Supplier hotel contracts: frontend implementation guide

Updated: 1 November 2026. This guide describes the checked-out backend, including Booking Confirmation Type and On Request SLA settings, release periods, booking cut-offs, cancellation policies, seasons, child pricing, restrictions and filters. Its PDF is generated from this Markdown file; keep examples and the PDF together when updating it. Audit baseline: f161d96. This expanded reference preserves current, stored-data and historical behavior.

Contents

0. Availability and reading map

1. Implemented API and access

2. Select the hotel, rooms and metadata

3. Complete base-model creation request

4. Complete fixed-model draft creation request

5. Children policies, booking restrictions and seasons

6. Complete contract update request

7. Full new-draft detail response

8. Contract lists, coverage and sorting

9. Errors, state and permissions

10. TypeScript integration and acceptance

11. Meal plans and recovered room-pricing reference

12. Complete fixed response and model update workflows

13. Complete picker and metadata responses

14. Field replacement and dependency matrices

15. Validation and failure scenario catalogue

16. Database invariants and frontend recovery

17. Complete field dictionary and schema coverage

18. Source and rule coverage matrix

19. Git history and content-loss audit

20. Current room/view and meal pricing

21. Contract inventory rules

22. Contract finalization

0. Availability and reading map

Current API means callable through one of the nine routes today. **Existing stored-data behavior** means retained readers/resolution and supported seasonal edits on already priced base contracts. **Removed API** means historical reference, not executable instructions. **Not implemented** means no working contract interface exists.

Capability	Availability and frontend implication
Base/fixed drafts	Current API for complete basics and optional sections, including seasonal child-price overrides
Meal/view metadata and view picker filter	Current API with localized labels/codes
Base room, supplements, meals, weekly pricing	Current POST/PATCH pricing section; complete scenarios in section 20
Seasonal room and meal overrides	Current POST/PATCH, validated against final pricing in the same request
Child bands/restrictions/cancellation	Current API stores/replaces configuration; no booking action/evaluation endpoint
Release/cut-off/SLA	Current API stores settings; inventory release, cut-off enforcement, timers, rejection/escalation execution not implemented here
Fixed room+meal+view pricing	Current POST/PATCH pricing.rooms, with per-variation meals
Inventory rule configuration	Current POST/PATCH inventoryRules; shared pool, per room/view and free sale; section 21
Finalization and nightly inventory	Current API: read-only review and atomic finalization; startup/minute lifecycle worker; published terms immutable
Daily overrides and booking	Not implemented; no inventory-calendar controls or booking execution

Use sections 1-10 for contract integration, 11 for historical pricing and migrated data, 12-16 for existing configuration scenarios, 17-19 for field coverage and history, and 20 for current room/view and meal pricing in both models.

1. Implemented API and access

All routes below start with `/suppliers/hotel-contracts`. Use `Authorization: Bearer <supplier_token>` and `Content-Type: application/json` for writes. Supplier super admins have the existing permission bypass; supplier admins need the indicated permission. Ownership and hotel eligibility always apply. IDs are positive JSON integers, at most 2147483647. Unknown fields, numeric strings in bodies and inappropriate nulls are rejected.

Method and suffix	Permission after suppliers.hotel_contracts.	Purpose
GET /metadata	metadata.view	Static meal plans and room views
GET /hotels	hotels.list	Eligible linked hotels
GET /hotels/:hotelId/rooms	rooms.list	Active rooms in an eligible hotel
POST /	create	Draft with optional child pricing, restrictions, seasons, cancellation, release and cut-off rules
GET /	list	Filtered supplier contracts, pagination and empty state
GET /:id	view	Full detail, sections, audit fields and version
PATCH /:id	update	Draft basics and full replacement of supplied sections
GET /:id/finalization-review	view	Completeness issues, confirmation and inventory preview
POST /:id/finalize	finalize	Atomic publication and nightly stock; HTTP 200

There are nine routes. Child pricing, restrictions, seasons, cancellation policies, release policies and booking cut-offs have no separate endpoints. Restriction activation is an `isActive` field within PATCH entries. Finalization publishes a complete Draft

and generates nightly inventory atomically (section 22); automatic status transitions follow the captured hotel timezone. New contracts are drafts; the stored status filter also supports active, paused, terminated, scheduled and expired without deriving status from coverage dates.

Both models accept optional pricing on contract POST/PATCH. The former PUT /:id/pricing remains removed. Canonical pricing replaces the historical basePricing response. Seasons can override room and meal prices against supplied or stored pricing; child prices remain independent. Inventory rules are supported through inventoryRules (section 21); finalization and nightly stock generation are implemented (section 22); daily rates and booking execution remain unavailable. Release periods and booking cut-offs remain configurable rules.

Responses use `{ status, message, data }`. Missing or another supplier's contract returns 404. Detail remains readable after a hotel or link becomes unavailable; writes recheck eligibility. Successful mutations return full detail and advance the shared version once; create starts at version 1.

2. Select the hotel, rooms and metadata

```
GET /suppliers/hotel-contracts/metadata
GET /suppliers/hotel-contracts/hotels?countryCode=EG&sortBy=nameEn&sortOrder=asc&page=1&limit=20
GET /suppliers/hotel-contracts/hotels/10/rooms?view=sea&minCapacity=2&maxCapacity=4&page=1&limit=20
```

Select real eligible IDs before submitting the examples. Hotels must be active with an approved, unsuspended supplier link; rooms must be active and belong to that hotel. Creation and updates recheck availability, so previously selected records can become unavailable.

Picker	Optional filters	sortBy values
Hotels	search, countryCode, city, starRating, propertyType	id, nameEn, nameAr, countryCode, city, starRating, createdAt
Rooms	search, view, bedTypeId, roomSizeId, minCapacity, maxCapacity	id, nameEn, nameAr, maxOccupancy, adults, areaSqm, createdAt

Both pickers accept page/limit and sortOrder=asc/desc. Defaults: page 1, limit 20 (maximum 100), nameEn ascending, nulls last, ID ascending tie-breaker unless sorting by ID itself. Text filters are literal case-insensitive substrings; hotel countryCode is normalized uppercase ISO alpha-2, starRating is 1..5. Capacity bounds compare maxOccupancy inclusively, accept 1..99, and require min <= max. Search lengths are at most 200; city at most 120. Unknown queries and invalid/repeated scalar values are rejected. Metadata and detail accept no query fields.

Picker data is `{ hotels: [...], meta, emptyState }` or `{ rooms: [...], meta, emptyState }`. Meta contains total, page, limit, totalPages. Empty states distinguish no_hotels/no_rooms from filtered no_results; null means no such empty state, even when the requested page is beyond the final page and total is positive. No matches gives totalPages=0. Reset page when filters change.

Metadata returns `mealPlans` and `roomViews`, each containing `{ value, labelEn, labelAr }` entries. Render localized labels and submit value codes:

- Meal plans: room_only, bed_and_breakfast, half_board, full_board, iftar_and_suhoor.

- Views: sea, city, garden, pool, mountain, courtyard, street, landmark, haram, partial_haram, no_view.

Hotel propertyType codes are hotel, resort, aparthotel, serviced_apartment, hostel, guesthouse, motel, villa and other.

Base meal supplements are shared across variations. Fixed meal supplements belong to each variation. Select explicit metadata views for contract variations; picker view filtering still uses the catalog view. Metadata adds mealPricingBases. See section 20 for complete pricing payloads.

3. Complete base-model creation request

`POST /suppliers/hotel-contracts` creates a draft atomically with any supplied child pricing, restrictions, seasons, cancellation policy, release policy and booking cut-off. The example includes those optional sections and omits pricing. Section 20 adds room/view and meal pricing to the same POST.

```
{
  "hotelId": 10,
  "name": "Winter 2026 contract",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "base_plus_supplements",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      }
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      }
    }
  ],
  "cancellationPolicy": {
```

```
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"}
}
```

Field	Validation and behavior
hotelId	Required eligible hotel ID; immutable after creation
name	Required trimmed nonblank string; at most 200 characters
startDate / endDate	Required real YYYY-MM-DD dates, years 0001..9999; start <= end; equal/past dates allowed
currency	Required ISO 4217 code, trimmed/uppercased; locks after room pricing is saved
bookingType	Required: on_request (On Request) or allotment_block (Allotment-block / Instant)
onRequestSettings	Optional complete SLA object; integer slaMinutes 10-120 and expiryAction auto_reject or escalate_to_hoteliana; null rejected on create
pricingModel	Required base_plus_supplements or fixed_per_room; immutable
childPricing	Optional; if supplied, all three categories are required. Omit it to create without child pricing; null is rejected
restrictions	Optional array, maximum 100; omit or [] for none
seasons	Optional array, maximum 100; omit or [] for none
cancellationPolicy	Optional complete tiers/noShow object; omit for none; null is rejected on create
releasePolicy	Optional complete release rule; omit for none; null is rejected on create
bookingCutoff	Optional complete cut-off rule; omit for none; null is rejected on create

A smaller valid create request omits child pricing and every optional section:

```
{
  "hotelId": 10,
  "name": "Winter 2026 draft without child pricing",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "pricingModel": "base_plus_supplements"
}
```

Its detail response has `onRequestSettings: null`, `childPricing: null`, `seasons: []`, `restrictions: []`, `cancellationPolicy: null`, `releasePolicy: null` and `bookingCutoff: null`. A later PATCH can add the complete child-pricing object; PATCH cannot clear it with null.

Do not send expectedVersion, status, supplierId, generated IDs or audits on create. Supplier identity comes from the session. Restriction isActive is not accepted on create; entries start active. A failed request rolls back all sections.

4. Complete fixed-model draft creation request

The fixed model accepts the same core fields and optional contract sections. This example deliberately omits pricing to demonstrate an unpriced draft; section 20 adds full fixed room/view rates and meal supplements.

```
{
  "hotelId": 10,
  "name": "Winter 2026 fixed draft",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "fixed_per_room",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      }
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
```

```
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      }
    }
  ],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"}
}
```

After HTTP 201, retain data.id and data.version. An unpriced draft returns weekPricing:null, pricing:null, pricingStatus:not_configured, pricingIssues:[] and resolvedRoomPrices:null. Supply pricing on a later PATCH to initialize it. Do not send the old fixedPricing/basePricing or top-level writable weekPricing fields.

5. Children policies, booking restrictions and seasons

Extra-child age bands and supplements

When supplied, childPricing has exactly three required categories: sharing_parents, shared_bed and extra_bed. Each has from, to and price. Ages are inclusive integer years 0..17, from <= to. Each category's upper age must also be <= the highest defined maxChildAge among this hotel's active rooms; the service falls back to 17 if no active room defines a cap. The examples assume the hotel permits children through age 11.

price is per child per night, in contract currency, from 0 through 10000000 with at most two decimal places. Zero means free. Each category is validated separately; the API does not require disjoint or gap-free bands across categories. Omit childPricing on create to save no child bands; detail then returns null. PATCH adds or replaces all three categories when supplied, leaves them unchanged when omitted, and rejects null or a partial object. There is no clear operation for saved child pricing.

Booking restrictions

Entry field	Rule
roomId	Optional active room from the contract hotel; omitted or null means all rooms
appliesFrom / appliesTo	Required real dates, ordered and inside the contract term when the section is written
minNights	Required integer 1..365
appliesOn	Required every_day or weekend_days
restrictedCheckinDays / restrictedCheckoutDays	Optional arrays of unique integers 0..6, maximum 7; omitted means []
isActive	PATCH entries only; boolean, default true; create always starts active

Restriction weekdays use 0=Sunday, 1=Monday, 2=Tuesday, 3=Wednesday, 4=Thursday, 5=Friday, 6=Saturday. Pricing weekendDays instead uses lowercase weekday names. appliesOn stores the intended daily/weekend scope; this draft API does not execute booking-time restriction resolution. Overlapping restrictions are accepted. Do not assume a booking-time winner or enforcement has been implemented.

PATCH replaces the whole supplied restriction array. Include all entries to retain, with isActive=false to disable an entry. Old rows are deleted and new IDs assigned; do not use a removed restriction toggle endpoint or treat old restriction IDs as stable. Omit restrictions to keep the existing set, or send [] to clear it. A term edit that omits restrictions does not rewrite/revalidate existing restriction rows; supply the section when adjusting their dates.

Rate seasons

A season contains name (trimmed nonblank, maximum 100), startDate, endDate and color (#RRGGBB normalized uppercase). Dates represent inclusive stay nights and must be ordered and within the contract term when seasons is supplied. No two seasons may share a night; a shared boundary date is an overlap. Adjacent seasons must start after the previous season's final night.

Validation reports every discovered season problem with its array path; overlap messages identify both seasons and their shared dates. Responses add id, contractId, inclusive nights, issues and audit fields. A season from 2026-12-20 through 2027-01-05 has 17 nights. Seasons are returned in date order.

Season pricing

A season can include `pricing.childPrices` to override any child category on its stay nights. Values are JSON numbers from 0 through 10000000 with at most two decimal places. The category's age band comes from contract `childPricing`; when child pricing is absent, `resolvedChildPricing` is null even if a season stores a child-price override. Omitted categories fall back to the contract price. When pricing is omitted, seasons return no room overrides and resolvedRoomPrices:null. Supply pricing in the same POST to add room/meal overrides.

Existing contracts are migrated to canonical pricing. Unknown meal prices/bases and unequal legacy supplements are exposed for review, with pricingStatus:incomplete and pricingIssues. Both new and existing drafts can save complete pricing through POST/PATCH. See section 20 for migration recovery and seasonal conflict handling.

PATCH replaces the whole supplied seasons array. Omit `seasons` to retain it; send `[]` to clear it. If a contract term is shortened while seasons are omitted, the edit succeeds and an out-of-term season returns an `outside_contract_period` issue. Surface issues for correction.

Cancellation policy

`cancellationPolicy` is optional on create (finalization requires effective coverage on every night) and is returned on every detail read; lists omit it.

Field	Rule
tiers	1..3 entries `{ daysBefore, charge }`; daysBefore is a whole number 1..365 and unique ("cancelled at least N days before check-in"). Returned sorted from the largest
noShow	Charge for a no-show, and for cancelling inside the smallest tier; always last
charge.kind	`free` (no value), `nights` (whole 1..365), `percent` (>0..100 of the stay; 100 = full stay) or `fixed` (>0..10000000 in the contract currency)

PATCH replaces the policy, omission keeps it and `null` removes it ("None"). Each season may also carry its own `cancellationPolicy`, which wins on that season's nights and is replaced together with the season. Every season response adds `cancellationPolicy` (its own or null), `effectiveCancellationPolicy` (season policy, else contract policy, else null) and `cancellationPolicySource` (`season`, `contract` or null). Do not send null for a seasonal policy. To remove a seasonal override, replace the full seasons array with that season's cancellationPolicy omitted; preserve its other fields and all other seasons. All charge values are strict JSON numbers with at most two decimals; free forbids value, including zero. For thresholds 7/free and 3/one night, cancellation at least 7 days ahead is free, 3 through 6 days costs one night, and inside 3 days uses noShow. These endpoints configure policies and expose inheritance; they do not quote a booking cancellation charge.

Errors carry their path, for example `seasons[0].cancellationPolicy.tiers.1.charge.value must be a whole number of nights from 1 to 365`.

Booking Confirmation Type and On Request SLA

Display `bookingType` as **Booking Confirmation Type**: `allotment_block` is **Allotment-block (Instant)** and `on_request` is **On Request**. These values remain separate from the hotel-linking confirmation setting.

Show SLA controls when the booking type is On Request, `releasePolicy.afterReleaseAction = on_request`, or `inventoryRules.whenSoldOut.action = on_request`. Free sale alone does not enable SLA settings. Label `slaMinutes` **SLA time (minutes)** and offer a whole-number input with min 10 and max 120. Label `expiryAction` **Action when the SLA expires**, with **Auto-reject** (`auto_reject`) and **Escalate to Hoteliana** (`escalate_to_hoteliana`). Both fields are required when configuring settings, but the complete settings object is optional in drafts. Timers, rejection, and escalation are not executed by these endpoints.

The type is `{ slaMinutes: number; expiryAction: "auto_reject" | "escalate_to_hoteliana" }`. Create accepts omission but rejects null. PATCH omission preserves settings, an object fully replaces them, and null clears them. List and detail always return `onRequestSettings` as the complete object or null. Existing contracts have null until configured.

```
{
  "expectedVersion": 1,
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 10, "expiryAction": "auto_reject"}
}
```

An instant contract can configure the release switch and its SLA in one PATCH:

```
{
  "expectedVersion": 2,
  "bookingType": "allotment_block",
  "releasePolicy": {"type": "same_day", "releaseTime": "18:00", "afterReleaseAction": "on_request"},
  "onRequestSettings": {"slaMinutes": 120, "expiryAction": "escalate_to_hoteliana"}
}

{"expectedVersion": 3, "onRequestSettings": null}
```

Eligibility uses the merged booking type, release policy and inventory sold-out policy, including saved values when omitted. Switching to instant preserves settings if a release or sold-out policy still allows On Request. If none of these paths allows On Request, saved settings are automatically cleared. Explicit non-null settings in that incompatible request return 400. Apply the returned detail and version to the form after every save.

Display validation messages such as **Enter a whole number from 10 to 120 minutes**, **Choose an action when the SLA expires**, and **Configure both SLA time and expiry action**. Preserve input after 400 errors. An incompatible selection returns `onRequestSettings requires an On Request booking, release or sold-out policy`. Do not send empty objects, numeric strings, partial objects, or null members.

Release period and booking cut-off

`releasePolicy` and `bookingCutoff` are independent, optional contract-level objects. Configure either one without the other. These endpoints store and return the rules; they do not expose an inventory release job or booking operation. Times are local 24-hour `HH:MM` strings from `00:00` through `23:59`; send no timezone or seconds. Responses use the same `HH:MM` format.

Field	Validation and behavior
releasePolicy.type	`same_day` or `number_of_days`
releasePolicy.daysBeforeCheckIn	Required integer 1..365 for `number_of_days`; omit for `same_day`. A `same_day` response returns null here
releasePolicy.releaseTime	Required local time of release, `HH:MM`
releasePolicy.afterReleaseAction	Required `stop_sale` or `on_request`
bookingCutoff.daysBeforeCheckIn	Required integer 0..365; 0 means the check-in day
bookingCutoff.cutoffTime	Required latest local booking time on that day, `HH:MM`

The full create examples use a seven-day release at 18:00 with `stop_sale` and a separate booking cut-off one day before check-in at 14:00. Omit either object on create to leave it unset; `null` is rejected on create. PATCH replaces either whole object when supplied, retains it when omitted, and clears it with `null`. `expectedVersion` applies to both, as it does to other edits. Detail and list rows always include `releasePolicy` and `bookingCutoff`, each null when unset. Do not derive one rule from the other or assume a required ordering between their days and times.

6. Complete contract update request

`PATCH /suppliers/hotel-contracts/501`. This independent example assumes current version 1; use the version actually returned by the latest read/write. It includes every mutable top-level field, including the release and cut-off rules, and disables the retained restriction.

```
{
  "expectedVersion": 1,
  "name": "Winter 2026 revised contract",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6],
      "isActive": false
    }
```

```
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E"
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      }
    }
  ],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"}
}
```

PATCH requires expectedVersion plus at least one mutable field, including optional pricing. Hotel, pricing model and status remain immutable. Supplied pricing fully replaces room/view rates, meals and weekly settings; omit it to preserve. Other section replacement rules remain unchanged. Dates validate against the merged term. Currency cannot change after pricing is saved.

A clear-only update is supported:

```
{
  "expectedVersion": 2,
  "restrictions": [],
  "seasons": [],
  "cancellationPolicy": null,
  "releasePolicy": null,
  "bookingCutoff": null
}
```

A `same_day` release uses no `daysBeforeCheckIn`. For example, this separate PATCH configures release at 18:00 on check-in day and a cut-off at 14:00 on check-in day:

```
{
  "expectedVersion": 2,
```

```
  "releasePolicy": {"type": "same_day", "releaseTime": "18:00", "afterReleaseAction": "on_request"},
  "bookingCutoff": {"daysBeforeCheckIn": 0, "cutoffTime": "14:00"}
}
```

All editable sections share one parent version. Serialize UI saves. After a 409 reload detail, preserve edits, show differences and reapply deliberately. Never guess the next version.

7. Full new-draft detail response

GET detail and successful POST/PATCH return the same detail shape. The following unpriced example has weekPricing:null, pricing:null, pricingStatus:not_configured, pricingIssues:[] and resolvedRoomPrices:null. Season cancellation and child-price resolution remain present. See section 20 for priced details.


```
{
  "status": "success",
  "message": "Contract draft created successfully",
  "data": {
    "hotelId": 10,
    "name": "Winter 2026 contract",
    "startDate": "2026-11-01",
    "endDate": "2027-03-31",
    "currency": "EGP",
    "bookingType": "on_request",
    "pricingModel": "base_plus_supplements",
    "id": 501,
    "supplierId": 7,
    "status": "draft",
    "version": 1,
    "weekPricing": null,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z",
    "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
    "childPricing": {
      "sharing_parents": {"from": 0, "to": 5, "price": 0},
      "shared_bed": {"from": 6, "to": 11, "price": 50},
      "extra_bed": {"from": 6, "to": 11, "price": 100}
    },
    "cancellationPolicy": {
      "tiers": [
        {
          "daysBefore": 7,
          "charge": {"kind": "free"}
        },
        {
          "daysBefore": 3,
          "charge": {"kind": "nights", "value": 1}
        }
      ],
      "noShow": {"kind": "percent", "value": 100}
    },
    "releasePolicy": {
      "type": "number_of_days",
      "daysBeforeCheckIn": 7,
      "releaseTime": "18:00",
      "afterReleaseAction": "stop_sale"
    },
    "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
    "restrictions": [
      {
        "appliesFrom": "2026-11-01",
        "appliesTo": "2027-03-31",
        "minNights": 2,
        "appliesOn": "weekend_days",
        "restrictedCheckinDays": [0, 1],
```

```
      "restrictedCheckoutDays": [6],
      "roomId": null,
      "isActive": true,
      "id": 601,
      "contractId": 501,
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T09:00:00.000Z",
      "updatedAt": "2026-10-27T09:00:00.000Z"
    },
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6],
      "isActive": true,
      "id": 600,
      "contractId": 501,
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T09:00:00.000Z",
      "updatedAt": "2026-10-27T09:00:00.000Z"
    }
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "rooms": [],
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      },
      "id": 700,
      "contractId": 501,
      "nights": 17,
      "issues": [],
      "resolvedRoomPrices": null,
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 120}
      },
      "cancellationPolicy": null,
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 7,
            "charge": {"kind": "free"}
          },
          {
            "daysBefore": 3,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "cancellationPolicySource": "contract",
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T09:00:00.000Z",
      "updatedAt": "2026-10-27T09:00:00.000Z"
    },
```

```
      {
        "name": "February winter season",
        "startDate": "2027-02-01",
        "endDate": "2027-02-28",
        "color": "#2E86AB",
        "cancellationPolicy": {
          "tiers": [
            {
              "daysBefore": 14,
              "charge": {"kind": "nights", "value": 1}
            }
          ],
          "noShow": {"kind": "percent", "value": 100}
        },
        "pricing": {
          "rooms": [],
          "childPrices": {}
        },
        "id": 701,
        "contractId": 501,
        "nights": 28,
        "issues": [],
        "resolvedRoomPrices": null,
        "resolvedChildPricing": {
          "sharing_parents": {"from": 0, "to": 5, "price": 0},
          "shared_bed": {"from": 6, "to": 11, "price": 50},
          "extra_bed": {"from": 6, "to": 11, "price": 100}
        },
        "effectiveCancellationPolicy": {
          "tiers": [
            {
              "daysBefore": 14,
              "charge": {"kind": "nights", "value": 1}
            }
          ],
          "noShow": {"kind": "percent", "value": 100}
        },
        "cancellationPolicySource": "season",
        "createdBySupplierUserId": 22,
        "updatedBySupplierUserId": 22,
        "createdAt": "2026-10-27T09:00:00.000Z",
        "updatedAt": "2026-10-27T09:00:00.000Z"
      }
    ],
    "pricing": null,
    "pricingStatus": "not_configured",
    "pricingIssues": [],
    "resolvedRoomPrices": null,
    "inventoryStatus": "not_configured",
    "inventoryRules": null,
    "inventoryIssues": [],
    "publicationStatus": "not_finalized",
    "publication": null
  }
}
```


Generated IDs and audit fields are response-only. On a section replacement, use the returned sections and version as the saved state. Serialize writable canonical pricing explicitly; never spread resolved prices, variationId, legacySupplement or pricing issues into POST/PATCH.

8. Contract lists, coverage and sorting

Query	Accepted values and semantics
hotelId / pricingModel	Exact eligible integer ID / base_plus_supplements or fixed_per_room
search / hotelName	Trimmed strings up to 200 characters; literal case-insensitive partial contract name / English or Arabic hotel name
status	draft, active, paused, terminated, scheduled, expired; stored status independent of dates
periods	Repeat for OR matching: everything, covering_today, starting_next_90_days, ending_next_90_days, already_over, custom
dateFrom / dateTo	Required real ordered YYYY-MM-DD dates with custom; forbidden without custom
sortBy	newest_first (default), ending_soonest, hotel_name
page / limit	Defaults 1 / 20; maximum limit 100

Different filters combine with AND. Periods combine with OR. Omitted periods or everything includes all coverage dates; custom still requires valid dateFrom/dateTo when present, even alongside everything. custom matches inclusive interval overlap. Today uses Asia/Riyadh; the next-90-day windows include today through day 90. covering_today means start <= today <= end; already_over means end < today.

Sorts: newest_first uses createdAt DESC, ID DESC; ending_soonest uses endDate ASC, ID ASC; hotel_name uses English hotel name ASC, nulls last, ID ASC. This endpoint has no sortOrder parameter. Date coverage never changes stored status.

```
GET /suppliers/hotel-contracts?periods=covering_today&periods=ending_next_90_days
  &sortBy=ending_soonest&page=1&limit=20
GET /suppliers/hotel-contracts?periods=custom&dateFrom=2026-12-01&dateTo=2026-12-31&hotelName=Nile
```

Lists include basics, audit/version, weekPricing, onRequestSettings, releasePolicy, bookingCutoff, hasPricing, pricingStatus and inventoryStatus. They omit pricing, pricingIssues, resolvedRoomPrices, inventoryRules, inventoryIssues and other detail sections. hasPricing is true for complete or incomplete configured pricing. A complete one-item list example is:


```
{
  "status": "success",
  "message": "Contract drafts retrieved successfully",
  "data": {
    "contracts": [
      {
        "hotelId": 10,
        "name": "Winter 2026 contract",
        "startDate": "2026-11-01",
        "endDate": "2027-03-31",
        "currency": "EGP",
        "bookingType": "on_request",
        "pricingModel": "base_plus_supplements",
        "id": 501,
        "supplierId": 7,
        "status": "draft",
        "version": 1,
        "weekPricing": null,
        "createdBySupplierUserId": 22,
        "updatedBySupplierUserId": 22,
        "createdAt": "2026-10-27T09:00:00.000Z",
        "updatedAt": "2026-10-27T09:00:00.000Z",
        "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
        "releasePolicy": {
          "type": "number_of_days",
          "daysBeforeCheckIn": 7,
          "releaseTime": "18:00",
          "afterReleaseAction": "stop_sale"
```

```
        },
        "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
        "hasPricing": false,
        "pricingStatus": "not_configured",
        "inventoryStatus": "not_configured",
        "publicationStatus": "not_finalized",
        "publication": null
      }
    ],
    "meta": {
      "total": 1,
      "page": 1,
      "limit": 20,
      "totalPages": 1
    },
    "emptyState": null
  }
}
```


emptyState=no_contracts means no supplier contracts before optional filters; no_results means filters match nothing; null is otherwise used, including out-of-range pages with a positive matching total. Keep the existing list row while detail loads and fetch full sections only from detail.

9. Errors, state and permissions

HTTP	Meaning	Frontend action
400	Invalid/unknown fields, money, selections or date/season rules	Display data.errors when present; keep form input
401	Missing, invalid or inactive supplier session	Restore authentication; reconcile uncertain writes
403	Role or exact endpoint permission denied	Disable unavailable actions and explain permission
404	Missing/cross-supplier contract or unavailable hotel/link	Show generic unavailable feedback and preserve edits
409	Stale expectedVersion or exhausted version counter	Reload and review conflicts; do not blindly retry
415	Wrong write Content-Type	Send application/json
5xx / offline	Server or transport failure	Preserve edits and reconcile before resending a write


```
{
  "status": "error",
  "message": "childPricing.shared_bed.from must be less than or equal to to",
  "data": {
    "errors": ["childPricing.shared_bed.from must be less than or equal to to"]
  }
}
```


Errors can also have data=null. data.errors is an array of strings, not a stable structured field/code map; show a form summary for messages you cannot map to fields. Guard against non-JSON transport responses. After a write timeout, GET detail and reconcile before resubmitting; no create idempotency key is implemented.

Maintain saved detail/version, editable form values, dirty fields, loading/saving flags, field/form errors and any conflict record separately. Do not spread detail responses into requests. Retain selections across picker pages, clear caches when the supplier account changes, cancel stale searches, and disable duplicate saves. Preserve untouched sections by omission.

10. TypeScript integration and acceptance

Companion TypeScript files beside the PDF generation source define creation/update and read shapes plus a fetch helper. Legacy pricing types are read-only. queryString accepts array values and appends repeated periods rather than joining them or replacing the previous value:

```
const query = queryString({
  periods: ["covering_today", "ending_next_90_days"],
  sortBy: "ending_soonest", page: 1, limit: 20
});
const draft = await api<ContractDetail>("", "POST", createBody);
const saved = await api<ContractDetail>(`/${draft.id}`, "PATCH", {
  expectedVersion: draft.version,
  onRequestSettings: { slaMinutes: 30, expiryAction: "auto_reject" },
  restrictions: [{ ...restrictionForm, isActive: false }]
});
// Replace the saved detail and shared version after every success.
```

Acceptance checklist:

- Test SLA bounds 10/120, both actions, invalid inputs, optional drafts, PATCH replace/retain/clear, release-switch eligibility, automatic clearing and list/detail consistency.

- Verify both create models work with childPricing omitted and return childPricing:null. When supplied, all three categories are required; invalid ages, hotel ceilings, prices, null and partial objects fail without partial writes. PATCH can add or replace, but cannot clear child pricing.

- Replace/retain/clear restrictions and seasons; disable a restriction through PATCH; test all-room and room-specific scope.

- Reject overlapping seasons (including shared end/start dates), impossible dates and supplied seasons outside the term. Shorten a term without seasons and display outside_contract_period warnings.

- Confirm the pricing PUT remains absent; unpriced drafts return null pricing and not_configured status. Verify both models can save pricing through POST/PATCH and retain stable variation IDs.

- Test independent release and cut-off creation, omission, full replacement and null clearing. Reject release type/day mismatches, invalid times and out-of-range day counts; verify same_day returns daysBeforeCheckIn:null and list/detail include both fields.

- Test all cancellation charge kinds, missing/extra values, duplicate thresholds, tier count/ranges, decimal limits and noShow. Verify create omission, PATCH retain/replace/null removal, seasonal precedence and inheritance after removing an override or the contract policy.

- Test seasonal child, room and meal overrides, independent inheritance and zero prices. Require pricing in the same request or saved on the contract, and require correction of incompatible retained overrides.

- Test currency locking, stale/concurrent writes, ownership, permissions, inactive rooms and unavailable hotel links.

- Test all list/picker filters, repeated periods, custom overlap, sort order, pagination and empty states. Test the Asia/Riyadh date boundary independently of the browser timezone.

- Confirm list/detail field differences for priced legacy contracts and new unpriced drafts. Both pricing models save through POST/PATCH; verify finalization review and atomic publication as described in section 22.

Build the PDF from this Markdown source with `python tmp/pdfs/hotel-contracts-frontend-v2/build_guide.py`. Keep the companion TypeScript examples aligned with the nine live routes. Contract database integration suites require `SUPPLIER_CONTRACTS_DB_TEST=1` and create disposable databases.

11. Meal plans and recovered room-pricing reference

Availability and meal-plan implementation

Existing stored-data behavior. Old meal selections are migrated with unknown costs and bases, which must be reviewed. **Current API:** GET /metadata supplies meal/view codes and mealPricingBases; POST/PATCH pricing saves actual meal supplements. Base meals are shared and fixed meals are scoped to a room/view variation.

Removed API. PUT /:id/pricing and its old basePricing body remain historical. Current writes use the pricing object described in section 20. Room only is implicit; the four other codes can carry per-person or per-room supplements.

Code	Meaning and pricing behavior
room_only	Room only; implicit zero supplement
bed_and_breakfast	Bed and breakfast upgrade
half_board	Half board upgrade
full_board	Full board upgrade
iftar_and_suhoor	Iftar and suhoor upgrade

Section 13 contains current English/Arabic labels. Canonical meal upgrades are sorted by code on read. Empty upgrade arrays are valid because room only is always available. The old requirement for at least one selected meal has been replaced.

Historical full pricing requests: do not execute today

These two alternatives describe the **removed API** `PUT /suppliers/hotel-contracts/501/pricing`. They are not steps in today's create workflow and must not be sent to POST/PATCH. Each independent historical example starts after base draft creation at version 1; its former successful write would return version 2.

All-week pricing uses amount only; weekendDays is forbidden, including an empty array.

```
{
  "expectedVersion": 1,
  "weekPricing": {"mode": "same_all_week"},
  "basePricing": {
    "baseRoomId": 101,
    "baseRate": {"amount": 1000},
    "mealPlans": ["room_only", "bed_and_breakfast"],
    "rooms": [
      {
        "roomId": 102,
        "supplement": {"amount": 200}
      }
    ]
  }
}
```

Weekday/weekend pricing requires 1-6 unique lowercase weekday names and both weekday and weekend for every base/supplement rate. The returned names are sorted Monday through Sunday.

```
{
  "expectedVersion": 1,
  "weekPricing": {
    "mode": "weekday_weekend",
    "weekendDays": ["friday", "saturday"]
  },
  "basePricing": {
    "baseRoomId": 101,
    "baseRate": {"weekday": 1000, "weekend": 1200},
```

```
    "mealPlans": ["half_board"],
    "rooms": [
      {
        "roomId": 102,
        "supplement": {"weekday": 200, "weekend": 300}
      }
    ]
  }
}
```

Field	Historical write / retained data rules
expectedVersion	Required integer 1..2147483647; stale version 409; parent version advanced once
weekPricing.mode	same_all_week or weekday_weekend; contract-wide, not per room
weekPricing.weekendDays	Required for split mode; 1..6 unique names monday..sunday. Omit for same mode; no seven-day weekend
basePricing.baseRoomId	Required active room of the selected hotel; unique base room
basePricing.baseRate	Complete rate in the selected weekly layout
basePricing.mealPlans	Required 1..5 unique metadata codes; no default or price per meal
basePricing.rooms	Required array of 0..1000 additional room supplements; [] means only the base room
rooms[].roomId	Active same-hotel room; unique and different from the base room
rooms[].supplement	Nonnegative complete rate in the same layout; additional cost, not an absolute replacement
Rate amount(s)	Finite JSON numbers 0..10000000 inclusive, at most two decimals; numeric strings/null/mixed layouts fail
Calculated totals	Base + supplement must also be <=10000000; zero is valid; negative discounts are not accepted

Historically the PUT fully replaced base prices and meal selections. Its DTO remains reference only. Current POST/PATCH pricing supports both models, actual meal supplements and explicit room/view pairs; see section 20.

Worked calculations and exact money handling

All-week example: room 101 = EGP 1000/night; room 102 = 1000 + 200 = EGP 1200/night. Split example: room 101 weekday/weekend = 1000/1200; room 102 = (1000+200)/(1200+300) = 1200/1500. Prices are per room per stay night, not per guest. Child additions are separate. The module returns nightly rates, not a booking-total or tax/fee breakdown.

Arithmetic uses integer hundredths and divides once: 0.10 + 0.20 = 0.30. Storage is numeric(12,2); response amounts are JSON numbers, not formatted strings. Format two decimals for display and submit numbers. Never add a supplement twice. A base of 9999999.99 plus 0.01 is valid; plus 0.02 exceeds the maximum. The base room has no additional supplement.

Complete existing priced detail example

Existing stored-data behavior. This complete response describes a readable historical base contract, not a result achievable by a new current API create. It has Friday/Saturday weekends and half_board. Its first season overrides room/child prices; its other season inherits room prices and overrides cancellation.

```
{
  "status": "success",
  "message": "Contract draft retrieved successfully",
  "data": {
    "hotelId": 10,
    "name": "Winter 2026 contract",
    "startDate": "2026-11-01",
    "endDate": "2027-03-31",
```

```
"currency": "EGP",
"bookingType": "on_request",
"pricingModel": "base_plus_supplements",
"id": 501,
"supplierId": 7,
"status": "draft",
"version": 2,
"weekPricing": {
  "mode": "weekday_weekend",
  "weekendDays": ["friday", "saturday"]
},
"createdBySupplierUserId": 22,
"updatedBySupplierUserId": 22,
"createdAt": "2026-10-27T09:00:00.000Z",
"updatedAt": "2026-10-27T09:00:00.000Z",
"onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
"childPricing": {
  "sharing_parents": {"from": 0, "to": 5, "price": 0},
  "shared_bed": {"from": 6, "to": 11, "price": 50},
  "extra_bed": {"from": 6, "to": 11, "price": 100}
},
"cancellationPolicy": {
  "tiers": [
    {
      "daysBefore": 7,
      "charge": {"kind": "free"}
    },
    {
      "daysBefore": 3,
      "charge": {"kind": "nights", "value": 1}
    }
  ],
  "noShow": {"kind": "percent", "value": 100}
},
"releasePolicy": {
  "type": "number_of_days",
  "daysBeforeCheckIn": 7,
  "releaseTime": "18:00",
  "afterReleaseAction": "stop_sale"
},
"bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
"restrictions": [
  {
    "appliesFrom": "2026-11-01",
    "appliesTo": "2027-03-31",
    "minNights": 2,
    "appliesOn": "weekend_days",
    "restrictedCheckinDays": [0, 1],
    "restrictedCheckoutDays": [6],
    "roomId": null,
    "isActive": true,
    "id": 601,
    "contractId": 501,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
  },
  {
    "roomId": 102,
    "appliesFrom": "2026-12-20",
    "appliesTo": "2027-01-05",
    "minNights": 2,
    "appliesOn": "every_day",
    "restrictedCheckinDays": [5],
    "restrictedCheckoutDays": [6],
    "isActive": true,
```

```
      "id": 600,
      "contractId": 501,
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T09:00:00.000Z",
      "updatedAt": "2026-10-27T09:00:00.000Z"
    }
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "baseRate": {"weekday": 1500, "weekend": 1800},
        "rooms": [
          {
            "roomId": 102,
            "view": "city",
            "legacySupplement": {"weekday": 250, "weekend": 350}
          }
        ],
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      },
      "id": 700,
      "contractId": 501,
      "nights": 17,
      "issues": [],
      "resolvedRoomPrices": [
        {
          "variationId": 1,
          "roomId": 101,
          "view": "city",
          "price": {"weekday": 1500, "weekend": 1800},
          "mealPlans": [
            {"type": "half_board", "pricingBasis": null, "price": null}
          ]
        },
        {
          "variationId": 2,
          "roomId": 102,
          "view": "city",
          "price": {"weekday": 1750, "weekend": 2150},
          "mealPlans": [
            {"type": "half_board", "pricingBasis": null, "price": null}
          ]
        }
      ],
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 120}
      },
      "cancellationPolicy": null,
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 7,
            "charge": {"kind": "free"}
          },
          {
            "daysBefore": 3,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
```

```
    },
    "cancellationPolicySource": "contract",
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
  },
  {
    "name": "February winter season",
    "startDate": "2027-02-01",
    "endDate": "2027-02-28",
    "color": "#2E86AB",
    "cancellationPolicy": {
      "tiers": [
        {
          "daysBefore": 14,
          "charge": {"kind": "nights", "value": 1}
        }
      ],
      "noShow": {"kind": "percent", "value": 100}
    },
    "pricing": {
      "rooms": [],
      "childPrices": {}
    },
    "id": 701,
    "contractId": 501,
    "nights": 28,
    "issues": [],
    "resolvedRoomPrices": [
      {
        "variationId": 1,
        "roomId": 101,
        "view": "city",
        "price": {"weekday": 1000, "weekend": 1200},
        "mealPlans": [
          {"type": "half_board", "pricingBasis": null, "price": null}
        ]
      },
      {
        "variationId": 2,
        "roomId": 102,
        "view": "city",
        "price": {"weekday": 1200, "weekend": 1500},
        "mealPlans": [
          {"type": "half_board", "pricingBasis": null, "price": null}
        ]
      }
    ],
    "resolvedChildPricing": {
      "sharing_parents": {"from": 0, "to": 5, "price": 0},
      "shared_bed": {"from": 6, "to": 11, "price": 50},
      "extra_bed": {"from": 6, "to": 11, "price": 100}
    },
    "effectiveCancellationPolicy": {
      "tiers": [
        {
          "daysBefore": 14,
          "charge": {"kind": "nights", "value": 1}
        }
      ],
      "noShow": {"kind": "percent", "value": 100}
    },
    "cancellationPolicySource": "season",
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
```

```
      "updatedAt": "2026-10-27T09:00:00.000Z"
    }
  ],
  "resolvedRoomPrices": [
    {
      "variationId": 1,
      "roomId": 101,
      "view": "city",
      "price": {"weekday": 1000, "weekend": 1200},
      "mealPlans": [
        {"type": "half_board", "pricingBasis": null, "price": null}
      ]
    },
    {
      "variationId": 2,
      "roomId": 102,
      "view": "city",
      "price": {"weekday": 1200, "weekend": 1500},
      "mealPlans": [
        {"type": "half_board", "pricingBasis": null, "price": null}
      ]
    }
  ],
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "variationId": 1,
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 1000, "weekend": 1200}
    },
    "rooms": [
      {
        "variationId": 2,
        "roomId": 102,
        "view": "city",
        "legacySupplement": {"weekday": 200, "weekend": 300}
      }
    ],
    "mealPlans": [
      {"type": "half_board", "pricingBasis": null, "price": null}
    ]
  },
  "pricingStatus": "incomplete",
  "pricingIssues": [
    {
      "path": "pricing.mealPlans[0].price",
      "code": "meal_price_missing",
      "message": "Enter the migrated meal price; it is not assumed free"
    },
    {
      "path": "pricing.mealPlans[0].pricingBasis",
      "code": "meal_basis_missing",
      "message": "Select per_person or per_room"
    },
    {
      "path": "pricing.rooms[0].supplement",
      "code": "room_supplement_review_required",
      "message": "Enter one supplement all week; original split values are preserved"
    },
    {
      "path": "seasons[0].pricing.rooms[0].supplement",
      "code": "season_supplement_review_required",
      "message": "Enter one seasonal supplement all week; original split values are preserved"
```

```
      }
    ],
    "inventoryStatus": "not_configured",
    "inventoryRules": null,
    "inventoryIssues": [],
    "publicationStatus": "not_finalized",
    "publication": null
  }
}
```


Seasonal precedence, combinations and stale data

Current API: seasons can override supplied or stored room/view and meal prices in either model. Components inherit independently. Seasonal supplements are scalar for new base pricing. Duplicate room/view pairs, adding rooms/meals, changing pricing basis and incompatible layouts are rejected. See section 20 for complete atomic examples.

In the example, the season base is 1500/1800, room 102 supplement is 250/350 and room 102 final price is 1750/2150. Override only the base and room 102 inherits 200/300, yielding 1700/2100. Override only the supplement and the base stays 1000/1200, yielding 1250/1550. Outside seasons use the contract rates. Omitted child categories inherit contract prices; explicit zero means free; age bands always come from contract childPricing. Without contract child bands, resolvedChildPricing is null even when child overrides are stored.

No daily override write route exists. The generic resolver accepts priority levels, but current detail supplies season then base. Do not infer a Daily > Season > Base API from the diagram. The resolver does not select a season by booking/check-in date for a booking engine.

Migrated incompatible base-rate layouts are visible through pricingIssues and ignored during resolution until corrected. New replacements that invalidate a seasonal reference or layout are rejected unless corrected in the same PATCH. Migration refuses stale historical room references rather than dropping them. An out-of-term season still produces outside_contract_period and is not silently trimmed.

Fixed room pricing: current variation model

Current API: fixed rooms are distinct room/view variations with full room-only rates and meal supplements. Section 20 contains working create/update examples. **Not implemented:** daily overrides and booking quotes remain separate work. Finalization and nightly inventory generation are implemented in section 22.

12. Complete fixed response and model update workflows

Shared fixture and independent version sequences

Use hotel 10 (active with approved supplier link), room 101 (base candidate), room 102 (additional/restricted room), supplier 7 and supplier user 22. The active rooms permit child ages through 11. Replace sample IDs with real eligible IDs. Base create independently returns contract 501/version 1; fixed create returns 502/version 1. Both cover 2026-11-01 through 2027-03-31. Historical pricing alternatives in section 11 are a separate workflow.

Full base creation is section 3, with its complete response in section 7. Full fixed creation is section 4; its complete HTTP 201 response follows. Both examples omit optional pricing. Section 20 demonstrates creating pricing and applying seasonal room/meal overrides atomically.


```
{
  "status": "success",
  "message": "Contract draft created successfully",
  "data": {
    "hotelId": 10,
    "name": "Winter 2026 fixed draft",
    "startDate": "2026-11-01",
    "endDate": "2027-03-31",
    "currency": "EGP",
    "bookingType": "on_request",
    "pricingModel": "fixed_per_room",
    "id": 502,
```

```
"supplierId": 7,
"status": "draft",
"version": 1,
"weekPricing": null,
"createdBySupplierUserId": 22,
"updatedBySupplierUserId": 22,
"createdAt": "2026-10-27T09:00:00.000Z",
"updatedAt": "2026-10-27T09:00:00.000Z",
"onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
"childPricing": {
  "sharing_parents": {"from": 0, "to": 5, "price": 0},
  "shared_bed": {"from": 6, "to": 11, "price": 50},
  "extra_bed": {"from": 6, "to": 11, "price": 100}
},
"cancellationPolicy": {
  "tiers": [
    {
      "daysBefore": 7,
      "charge": {"kind": "free"}
    },
    {
      "daysBefore": 3,
      "charge": {"kind": "nights", "value": 1}
    }
  ],
  "noShow": {"kind": "percent", "value": 100}
},
"releasePolicy": {
  "type": "number_of_days",
  "daysBeforeCheckIn": 7,
  "releaseTime": "18:00",
  "afterReleaseAction": "stop_sale"
},
"bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
"restrictions": [
  {
    "appliesFrom": "2026-11-01",
    "appliesTo": "2027-03-31",
    "minNights": 2,
    "appliesOn": "weekend_days",
    "restrictedCheckinDays": [0, 1],
    "restrictedCheckoutDays": [6],
    "roomId": null,
    "isActive": true,
    "id": 611,
    "contractId": 502,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
  },
  {
    "roomId": 102,
    "appliesFrom": "2026-12-20",
    "appliesTo": "2027-01-05",
    "minNights": 2,
    "appliesOn": "every_day",
    "restrictedCheckinDays": [5],
    "restrictedCheckoutDays": [6],
    "isActive": true,
    "id": 610,
    "contractId": 502,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
  }
```

```
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "rooms": [],
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      },
      "id": 710,
      "contractId": 502,
      "nights": 17,
      "issues": [],
      "resolvedRoomPrices": null,
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 120}
      },
      "cancellationPolicy": null,
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 7,
            "charge": {"kind": "free"}
          },
          {
            "daysBefore": 3,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "cancellationPolicySource": "contract",
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T09:00:00.000Z",
      "updatedAt": "2026-10-27T09:00:00.000Z"
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "pricing": {
        "rooms": [],
        "childPrices": {}
      },
      "id": 711,
      "contractId": 502,
      "nights": 28,
      "issues": [],
      "resolvedRoomPrices": null,
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
```

```
          "extra_bed": {"from": 6, "to": 11, "price": 100}
        },
        "effectiveCancellationPolicy": {
          "tiers": [
            {
              "daysBefore": 14,
              "charge": {"kind": "nights", "value": 1}
            }
          ],
          "noShow": {"kind": "percent", "value": 100}
        },
        "cancellationPolicySource": "season",
        "createdBySupplierUserId": 22,
        "updatedBySupplierUserId": 22,
        "createdAt": "2026-10-27T09:00:00.000Z",
        "updatedAt": "2026-10-27T09:00:00.000Z"
      }
    ],
    "pricing": null,
    "pricingStatus": "not_configured",
    "pricingIssues": [],
    "resolvedRoomPrices": null,
    "inventoryStatus": "not_configured",
    "inventoryRules": null,
    "inventoryIssues": [],
    "publicationStatus": "not_finalized",
    "publication": null
  }
}
```


Full base update and response

Current API. PATCH /suppliers/hotel-contracts/501 using expectedVersion:1. Replaces optional sections, revises name/SLA action and disables the room-specific restriction. Successful response is version 2. Replacing restrictions/seasons changes row IDs; do not use those IDs as stable form identities.


```
{
  "expectedVersion": 1,
  "name": "Winter 2026 revised contract",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 120, "expiryAction": "escalate_to_hoteliana"},
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6],
      "isActive": false
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6],
```

```
      "isActive": true
    }
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      }
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      }
    }
  ],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"}
}

{
  "status": "success",
  "message": "Contract draft updated successfully",
  "data": {
    "hotelId": 10,
    "name": "Winter 2026 revised contract",
    "startDate": "2026-11-01",
    "endDate": "2027-03-31",
    "currency": "EGP",
    "bookingType": "on_request",
    "pricingModel": "base_plus_supplements",
    "id": 501,
    "supplierId": 7,
    "status": "draft",
    "version": 2,
    "weekPricing": null,
```

```
"createdBySupplierUserId": 22,
"updatedBySupplierUserId": 22,
"createdAt": "2026-10-27T09:00:00.000Z",
"updatedAt": "2026-10-27T10:00:00.000Z",
"onRequestSettings": {"slaMinutes": 120, "expiryAction": "escalate_to_hoteliana"},
"childPricing": {
  "sharing_parents": {"from": 0, "to": 5, "price": 0},
  "shared_bed": {"from": 6, "to": 11, "price": 50},
  "extra_bed": {"from": 6, "to": 11, "price": 100}
},
"cancellationPolicy": {
  "tiers": [
    {
      "daysBefore": 7,
      "charge": {"kind": "free"}
    },
    {
      "daysBefore": 3,
      "charge": {"kind": "nights", "value": 1}
    }
  ],
  "noShow": {"kind": "percent", "value": 100}
},
"releasePolicy": {
  "type": "number_of_days",
  "daysBeforeCheckIn": 7,
  "releaseTime": "18:00",
  "afterReleaseAction": "stop_sale"
},
"bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
"restrictions": [
  {
    "appliesFrom": "2026-11-01",
    "appliesTo": "2027-03-31",
    "minNights": 2,
    "appliesOn": "weekend_days",
    "restrictedCheckinDays": [0, 1],
    "restrictedCheckoutDays": [6],
    "isActive": true,
    "roomId": null,
    "id": 701,
    "contractId": 501,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T10:00:00.000Z",
    "updatedAt": "2026-10-27T10:00:00.000Z"
  },
  {
    "roomId": 102,
    "appliesFrom": "2026-12-20",
    "appliesTo": "2027-01-05",
    "minNights": 2,
    "appliesOn": "every_day",
    "restrictedCheckinDays": [5],
    "restrictedCheckoutDays": [6],
    "isActive": false,
    "id": 700,
    "contractId": 501,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T10:00:00.000Z",
    "updatedAt": "2026-10-27T10:00:00.000Z"
  }
],
"seasons": [
  {
    "name": "Christmas peak",
```

```
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "rooms": [],
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      },
      "id": 800,
      "contractId": 501,
      "nights": 17,
      "issues": [],
      "resolvedRoomPrices": null,
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 120}
      },
      "cancellationPolicy": null,
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 7,
            "charge": {"kind": "free"}
          },
          {
            "daysBefore": 3,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "cancellationPolicySource": "contract",
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T10:00:00.000Z",
      "updatedAt": "2026-10-27T10:00:00.000Z"
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "pricing": {
        "rooms": [],
        "childPrices": {}
      },
      "id": 801,
      "contractId": 501,
      "nights": 28,
      "issues": [],
      "resolvedRoomPrices": null,
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 100}
      },
      "effectiveCancellationPolicy": {
        "tiers": [
```

```
            {
              "daysBefore": 14,
              "charge": {"kind": "nights", "value": 1}
            }
          ],
          "noShow": {"kind": "percent", "value": 100}
        },
        "cancellationPolicySource": "season",
        "createdBySupplierUserId": 22,
        "updatedBySupplierUserId": 22,
        "createdAt": "2026-10-27T10:00:00.000Z",
        "updatedAt": "2026-10-27T10:00:00.000Z"
      }
    ],
    "pricing": null,
    "pricingStatus": "not_configured",
    "pricingIssues": [],
    "resolvedRoomPrices": null,
    "inventoryStatus": "not_configured",
    "inventoryRules": null,
    "inventoryIssues": [],
    "publicationStatus": "not_finalized",
    "publication": null
  }
}
```


Full fixed update and response

Current API. PATCH /suppliers/hotel-contracts/502 using expectedVersion:1. The same complete section semantics apply. Response version is 2. This does not save fixed room pricing.


```
{
  "expectedVersion": 1,
  "name": "Winter 2026 fixed revised contract",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 120, "expiryAction": "escalate_to_hoteliana"},
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6],
      "isActive": false
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6],
      "isActive": true
    }
  ],
  "seasons": [
    {
```

```
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      }
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      }
    }
  ],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"}
}

{
  "status": "success",
  "message": "Contract draft updated successfully",
  "data": {
    "hotelId": 10,
    "name": "Winter 2026 fixed revised contract",
    "startDate": "2026-11-01",
    "endDate": "2027-03-31",
    "currency": "EGP",
    "bookingType": "on_request",
    "pricingModel": "fixed_per_room",
    "id": 502,
    "supplierId": 7,
    "status": "draft",
    "version": 2,
    "weekPricing": null,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T10:00:00.000Z",
    "onRequestSettings": {"slaMinutes": 120, "expiryAction": "escalate_to_hoteliana"},
```

```
"childPricing": {
  "sharing_parents": {"from": 0, "to": 5, "price": 0},
  "shared_bed": {"from": 6, "to": 11, "price": 50},
  "extra_bed": {"from": 6, "to": 11, "price": 100}
},
"cancellationPolicy": {
  "tiers": [
    {
      "daysBefore": 7,
      "charge": {"kind": "free"}
    },
    {
      "daysBefore": 3,
      "charge": {"kind": "nights", "value": 1}
    }
  ],
  "noShow": {"kind": "percent", "value": 100}
},
"releasePolicy": {
  "type": "number_of_days",
  "daysBeforeCheckIn": 7,
  "releaseTime": "18:00",
  "afterReleaseAction": "stop_sale"
},
"bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
"restrictions": [
  {
    "appliesFrom": "2026-11-01",
    "appliesTo": "2027-03-31",
    "minNights": 2,
    "appliesOn": "weekend_days",
    "restrictedCheckinDays": [0, 1],
    "restrictedCheckoutDays": [6],
    "isActive": true,
    "roomId": null,
    "id": 711,
    "contractId": 502,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T10:00:00.000Z",
    "updatedAt": "2026-10-27T10:00:00.000Z"
  },
  {
    "roomId": 102,
    "appliesFrom": "2026-12-20",
    "appliesTo": "2027-01-05",
    "minNights": 2,
    "appliesOn": "every_day",
    "restrictedCheckinDays": [5],
    "restrictedCheckoutDays": [6],
    "isActive": false,
    "id": 710,
    "contractId": 502,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T10:00:00.000Z",
    "updatedAt": "2026-10-27T10:00:00.000Z"
  }
],
"seasons": [
  {
    "name": "Christmas peak",
    "startDate": "2026-12-20",
    "endDate": "2027-01-05",
    "color": "#E4572E",
    "pricing": {
      "rooms": [],
```

```
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      },
      "id": 810,
      "contractId": 502,
      "nights": 17,
      "issues": [],
      "resolvedRoomPrices": null,
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 120}
      },
      "cancellationPolicy": null,
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 7,
            "charge": {"kind": "free"}
          },
          {
            "daysBefore": 3,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "cancellationPolicySource": "contract",
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T10:00:00.000Z",
      "updatedAt": "2026-10-27T10:00:00.000Z"
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "pricing": {
        "rooms": [],
        "childPrices": {}
      },
      "id": 811,
      "contractId": 502,
      "nights": 28,
      "issues": [],
      "resolvedRoomPrices": null,
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 100}
      },
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
```

```
          "noShow": {"kind": "percent", "value": 100}
        },
        "cancellationPolicySource": "season",
        "createdBySupplierUserId": 22,
        "updatedBySupplierUserId": 22,
        "createdAt": "2026-10-27T10:00:00.000Z",
        "updatedAt": "2026-10-27T10:00:00.000Z"
      }
    ],
    "pricing": null,
    "pricingStatus": "not_configured",
    "pricingIssues": [],
    "resolvedRoomPrices": null,
    "inventoryStatus": "not_configured",
    "inventoryRules": null,
    "inventoryIssues": [],
    "publicationStatus": "not_finalized",
    "publication": null
  }
}
```


Follow-up branches: retain, clear and child-price override

Each request below is an independent branch starting from base draft 501/version 2; do not run the examples sequentially with the same version. Success returns version 3. For fixed draft 502 use its own latest version.

Omit optional sections to retain them:

```
{"expectedVersion": 2, "name": "Winter 2026 renamed only"}
```

Clear all currently clearable optional sections, keeping required basics and saved child bands:

```
{
  "expectedVersion": 2,
  "onRequestSettings": null,
  "restrictions": [],
  "seasons": [],
  "cancellationPolicy": null,
  "releasePolicy": null,
  "bookingCutoff": null
}
```

Replace seasons with a child-price override. The old February season is intentionally removed; include it in this replacement if it must remain.

```
{
  "expectedVersion": 2,
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "childPrices": {"extra_bed": 120, "sharing_parents": 0}
      }
    }
  ]
}
```

That saved season omits baseRate, has pricing.rooms:[], childPrices:{extra_bed:120,sharing_parents:0}, resolvedRoomPrices:null, and resolvedChildPricing with unchanged contract age bands and those overridden prices. It inherits contract cancellation because its own policy was omitted.

13. Complete picker and metadata responses

Full metadata and localized labels

Current API. GET /suppliers/hotel-contracts/metadata accepts no query. These exact labels come from CONTRACT_MEAL_PLANS and hotel-library ROOM_VIEWS. Persist value codes, never labels. Preserve Arabic in UTF-8; locale never changes the code.


```
{
  "status": "success",
  "message": "Contract metadata retrieved successfully",
  "data": {
    "mealPlans": [
      {"value": "room_only", "labelEn": "Room only", "labelAr": "إقامة فقط"},
      {"value": "bed_and_breakfast", "labelEn": "Bed and breakfast", "labelAr": "إقامة وإفطار"},
      {"value": "half_board", "labelEn": "Half board", "labelAr": "نصف إقامة"},
      {"value": "full_board", "labelEn": "Full board", "labelAr": "إقامة كاملة"},
      {"value": "iftar_and_suhoor", "labelEn": "Iftar and suhoor", "labelAr": "إفطار وسحور"}
    ],
    "roomViews": [
      {"value": "sea", "labelEn": "Sea view", "labelAr": "إطلالة على البحر"},
      {"value": "city", "labelEn": "City view", "labelAr": "إطلالة على المدينة"},
      {"value": "garden", "labelEn": "Garden view", "labelAr": "إطلالة على الحديقة"},
      {"value": "pool", "labelEn": "Pool view", "labelAr": "إطلالة على المسبح"},
      {"value": "mountain", "labelEn": "Mountain view", "labelAr": "إطلالة على الجبل"},
      {"value": "courtyard", "labelEn": "Courtyard view", "labelAr": "إطلالة على الفناء"},
      {"value": "street", "labelEn": "Street view", "labelAr": "إطلالة على الشارع"},
      {"value": "landmark", "labelEn": "Landmark view", "labelAr": "إطلالة على معلم بارز"},
      {
        "value": "partial_haram",
        "labelEn": "Partial Haram view",
        "labelAr": "إطلالة جزئية على الحرم"
      },
      {"value": "haram", "labelEn": "Haram view", "labelAr": "إطلالة على الحرم"},
      {"value": "no_view", "labelEn": "No view", "labelAr": "بدون إطلالة"}
    ],
    "mealPricingBases": ["per_person", "per_room"],
    "roomSizes": [],
    "bedTypes": [],
    "inventoryModes": [
      {"value": "shared_pool", "labelEn": "Shared pool", "labelAr": "مخزون مشترك"},
      {
        "value": "per_room_type",
        "labelEn": "Per room/view type",
        "labelAr": "لكل نوع غرفة وإطلالة"
      },
      {"value": "free_sale", "labelEn": "Free sale", "labelAr": "بيع حر"}
    ],
    "soldOutActions": [
      {"value": "stop_sale", "labelEn": "Stop sale", "labelAr": "إيقاف البيع"},
      {
        "value": "on_request",
        "labelEn": "Switch to On Request",
        "labelAr": "التحويل إلى عند الطلب"
      },
      {
        "value": "controlled_overbooking",
        "labelEn": "Controlled overbooking",
        "labelAr": "حجز زائد بحدود"
      }
    ]
  }
}
```

Full hotel picker response

GET /suppliers/hotel-contracts/hotels?countryCode=EG&page=1&limit=20. nameEn/nameAr/countryCode/city/starRating/propertyType can be null in other records. These are picker projections, not complete hotel-library records.


```
{
  "status": "success",
  "message": "Contract hotel options retrieved successfully",
  "data": {
    "hotels": [
      {
        "id": 10,
        "nameEn": "Nile Hotel",
        "nameAr": "فندق النيل",
        "countryCode": "EG",
        "city": "Cairo",
        "starRating": 5,
        "propertyType": "hotel",
        "status": "active"
      }
    ],
    "meta": {
      "total": 1,
      "page": 1,
      "limit": 20,
      "totalPages": 1
    },
    "emptyState": null
  }
}
```


Full room picker response

GET /suppliers/hotel-contracts/hotels/10/rooms?page=1&limit=20. Both sample rooms are shown with every returned field. Bed and size IDs refer to library records; their localized names/lookups are not supplied by this module. maxChildAge is used in service validation but is not in this picker response: handle server child-cap errors rather than assuming the field is returned.


```
{
  "status": "success",
  "message": "Contract room options retrieved successfully",
  "data": {
    "rooms": [
      {
        "id": 101,
        "hotelId": 10,
        "nameEn": "Standard City",
        "nameAr": "قياسية بإطلالة على المدينة",
        "view": "city",
        "maxOccupancy": 3,
        "adults": 2,
        "bedTypeId": 1,
        "roomSizeId": 1,
        "areaSqm": 25,
        "status": "active",
        "roomSizeCode": null,
        "bedTypeCode": null
      },
      {
        "id": 102,
        "hotelId": 10,
        "nameEn": "Deluxe Sea",
        "nameAr": "ديلوكس بإطلالة على البحر",
        "view": "sea",
        "maxOccupancy": 4,
```

```
        "adults": 2,
        "bedTypeId": 1,
        "roomSizeId": 2,
        "areaSqm": 35,
        "status": "active",
        "roomSizeCode": null,
        "bedTypeCode": null
      }
    ],
    "meta": {
      "total": 2,
      "page": 1,
      "limit": 20,
      "totalPages": 1
    },
    "emptyState": null
  }
}
```


Property type values: hotel, resort, aparthotel, serviced_apartment, hostel, guesthouse, motel, villa, other. View matches the metadata code exactly. BedTypeId/roomSizeId are equality filters. Capacity bounds compare maxOccupancy, not adults. Hotel city/name searches are literal case-insensitive substrings; country/type/stars are exact. Percent and underscore in searches are literal, not user wildcards.

Empty and page-boundary responses

No eligible hotels gives hotels:[], total:0, totalPages:0 and emptyState:no_hotels. Eligible hotel without active rooms uses no_rooms. Contracts use no_contracts when none exist for this supplier before optional filters. A nonempty underlying set with zero matches uses no_results. If total is positive but the page is beyond totalPages, the array is empty and emptyState is null: return to a valid page. Metadata has no pagination/emptyState.

14. Field replacement and dependency matrices

Field / section	Exact create and PATCH semantics
hotelId, pricingModel	Required on create; forbidden on PATCH; immutable
name, dates, currency, bookingType	Required on create, optional on PATCH; omit to preserve; null rejected. Currency locks after stored room pricing
expectedVersion	Forbidden on create; required on PATCH, together with at least one mutable field
onRequestSettings	Create omit=null, null rejected. PATCH omit preserves, object fully replaces, null clears. {} and partial pairs invalid
childPricing	Create omission=null; all three categories required if supplied. PATCH omit preserves; object fully replaces; null/{} invalid; no clear operation
restrictions	Create omission/[]=none. PATCH omission preserves, [] clears, array fully replaces. isActive permitted only on PATCH entries, default true
inventoryRules	Optional on create; POST null rejected. PATCH omit preserves, object replaces, null clears on drafts. Requires final pricing variation identities; missing controlled limits flag incomplete drafts
seasons	Create omission/[]=none. PATCH omission preserves, [] clears, array replaces all rows and nested supplements/policies
cancellationPolicy	Create omission=null, null rejected. PATCH omit preserves, null clears, complete object replaces only the contract policy
seasons[].cancellationPolicy	Omit from supplied season means inherit contract, not preserve the previous seasonal policy. null/{} invalid
seasons[].pricing	Omit or {}=no overrides; null invalid. Components inherit independently; no merge with the old season record
seasons[].pricing.childPrices	Every category optional; {}=no override; zero=free; null invalid
releasePolicy, bookingCutoff	Independently: create omit=null, null rejected; PATCH omit preserves, null clears, complete object replaces
restrictions[].roomId	Omit/null=all rooms; positive integer=specific active same-hotel room
Closed weekday arrays	Omit=[] on stored row; []=no closed weekdays; null invalid. Parent row replacement replaces arrays
pricing	Optional on POST/PATCH. Omit to preserve; supplied object fully replaces configuration; null invalid. Both models supported
weekPricing, basePricing, fixedPricing, mealPlans	Not accepted as top-level writable fields; weekly settings and meals belong inside pricing
IDs, audits, status, version, calculated fields	Server managed/read-only; never spread responses into write bodies

On Request dependency truth table

Resulting configuration	SLA settings behavior
bookingType=on_request, any release	Allowed, still optional in drafts
allotment_block + release afterReleaseAction=on_request	Allowed; preconfigures release switch; still optional
allotment_block + no release or sold-out request path	Explicit SLA object rejected 400; omitted saved settings automatically cleared
Instant + inventory whenSoldOut.action=on_request	SLA allowed through the inventory fallback; same shared SLA object
Switch type to instant, release remains on_request	Omitted existing settings retained
Remove release switch, type remains on_request	Omitted existing settings retained
Remove all On Request paths + null	Accepted and cleared
Remove all On Request paths + object	Rejected400; whole write rolls back
On Request + null	Accepted incomplete draft; no default minutes/action assigned

Eligibility uses merged stored/request bookingType, releasePolicy and inventoryRules.whenSoldOut. releaseTime is local HH:MM without a stored timezone; no scheduler executes it. Coverage filtering uses Asia/Riyadh, a separate calendar-date policy, not a release timezone.

Query combinations and defaults

Numeric query conversion accepts complete unsigned decimal strings only; whitespace, signs, fractions, booleans and repeated scalar arrays fail. Repeated periods is supported; comma-separated periods and periods[] are not this interface. Duplicate valid periods are accepted (uniqueness is not enforced). Text is trimmed; empty search/city/hotelName is valid with no effective filter. Empty enum/date/number values fail.

Period members combine with OR; other contract filters combine with AND. Omission/everything removes coverage filtering. custom still requires both dates when everything is selected; dateFrom/dateTo without custom fails 400. covering_today includes both ends; next90day ranges include today and today+90; already_over means endDate<today; custom means inclusive interval overlap, not containment. Calendar today is Asia/Riyadh regardless of browser/server timezone.

Contract sorts: newest_first = createdAt DESC then id DESC; ending_soonest = endDate ASC then id ASC; hotel_name = English hotel name ASC NULLS LAST then id ASC. Contract-list sortOrder is rejected. Pickers default nameEn ASC NULLS LAST then id ASC; asc/desc allowed for picker sort fields. Sorting by id has no extra tie-breaker. page defaults 1 (max 2147483647), limit defaults 20 (max 100), minimum 1 for both. Full filter fields are in section 17.

Status and activation boundary

Stored statuses: draft, active, paused, terminated, scheduled, expired. Create always saves Draft. PATCH rejects status and non-Draft edits. Finalization atomically publishes Active or Scheduled after validation and stock generation. The lifecycle worker advances or expires published contracts using the captured hotel timezone; list coverage filters still use Asia/Riyadh. Inventory null clearing and incomplete inventory replacements are Draft-only. See section 22 for readiness and retry behavior.

15. Validation and failure scenario catalogue

These tables use abbreviated inputs to expose boundaries; the complete bodies are in preceding sections. Current rules apply unless explicitly marked historical. A structurally valid request can still fail ownership/eligibility. Error ordering and exact text are not stable machine interfaces.

Area / input	Expected outcome
Body hotelId:"10", slaMinutes:"30", price:"50"	400; JSON numeric strings are not coerced
POST status/expectedVersion/basePricing; PATCH hotelId/pricingModel	400; strict allowlist rejects unknown/read-only fields
ID0,-1,1.5,2147483648	400; IDs are positive32bit integers. Valid shape still requires eligible record
Nondecimal route ID or out-of-range /:hotelId	400 from DatabaseIdPipe
Name whitespace-only or >200chars	400; accepted contract names are trimmed1..200
Date2026-02-29,0000-01-01,timestamp	400; real YYYY-MM-DD, years0001..9999
Reversed term vs same-day/past valid term	Reversed400; equal dates and valid past dates accepted
Currency" egp " vs EG,ZZZ,null	EGP normalized/accepted; malformed/unknown/null400
Change currency with stored room pricing	400; unchanged currency and unpriced edits accepted
Omit every optional create section	Accepted draft; nullable sections null, arrays[]
PATCH expectedVersion only	400; at least one mutable field required
Stale version / max current version 2147483647	409; no writes; reload/reconcile
bookingType instant,On Request,null	400; only allotment_block/on_request
SLA10/120 vs9/121/10.5/null/unknown action	Inclusive bounds accepted, invalid 400
SLA{}, missing member, extra field	400; complete eligible pair required
SLA create null vs PATCH null	Create400; PATCH clears/advances version
childPricing partial,{},null	400 if supplied; complete omission on create allowed
Child ages0..17,from<=to vs18/fraction/reversed	Valid accepted; invalid 400
Child to above highest defined active-room maxChildAge	400; fallback17 if no active room defines cap
Overlapping child category bands	Accepted; no disjoint/gap-free requirement across categories
Child price0/10000000 vs-0.01/10000000.01/1.001	Bounds valid; negative/overflow/>2decimals400

Area / input	Expected outcome
PATCH childPricing:null	400; no child-band clear operation
Restriction roomId omitted/null/active same-hotel room	Accepted; stored null=all rooms
Restriction room inactive/missing/wrong hotel	400 with nested field path
Restrictions0/100 entries vs101	Valid counts accepted;101 returns 400
Restriction dates reversed/outside merged term	400; supplied row must fit contract
Shorten term, omit restrictions	Saved restrictions retained without term revalidation
minNights 1/365 vs0/366/fraction	Inclusive bounds accepted; invalid 400
Closed weekdays[] or unique0..6 vs7/duplicates/null	Valid accepted; invalid 400; max 7items
isActive on create vs update restriction	POST400; PATCH boolean allowed, omitted defaults true
Overlapping restrictions	Stored; no booking-time winner/enforcement implemented
Seasons[]/100 vs101	Valid counts accepted;101 returns 400
Season name blank/>100chars or color#FFF	400; trimmed1..100name and full#RRGGBB required; color uppercase normalized
Season same start/end vs reversed/outside term	One inclusive night accepted; invalid 400
SeasonA endsJan05 and B startsJan05	400 shared night; startingJan06 can be valid
Shorten term, omit seasons	Accepted; detail reports outside_contract_period
childPrices:{extra_bed:0} without room pricing	Accepted; resolution null if no contract child bands
Seasonal baseRate/nonempty rooms on unpriced draft	400; saved contract room pricing prerequisite
pricing:{}, rooms:[], childPrices:{}	Accepted no overrides; null objects/members invalid
Seasonal duplicate/base/new supplement room	400; only existing contract supplement rooms allowed
amount rate for saved weekday_weekend mode	400; must use complete matching layout
Seasonal calculated price>10000000	400; whole update rolls back
Cancellation1..3unique thresholds 1..365 + noShow	Complete policy accepted; invalid count/range/duplicate400
kind:free with value omitted vs0/null	Omission accepted; any supplied value invalid
kind:nights value 1/365 vs0/1.5/366	Bounds accepted; invalid 400

Area / input	Expected outcome
kind:percent value 0.01/100 vs0/100.01	Valid within2decimals; invalid 400
kind:fixed value 0.01/10000000 vs0/overflow	Valid within2decimals; invalid 400
Missing noShow or missing nonfree charge.value	400; noShow also covers inside smallest tier
Unsorted unique cancellation thresholds	Accepted; returned largest daysBefore first
Seasonal policy supplied/omitted/null	Own policy wins/contract inherited/400
PATCH contract cancellationPolicy:null	Clears contract only; seasonal own policies remain
same_day release with daysBeforeCheckIn0/null/1	400; omit that field entirely
number_of_days release1/365 vs missing/0/366	Bounds accepted; invalid 400
Time00:00/23:59 vs24:00/9:00/18:00:00/timezone	Only exact HH:MM accepted
Release stop_sale/on_request vs unknown	Known values only; on_request permits SLA configuration
Cut-off days 0/365 vs-1/366/fraction	Bounds accepted;0=check-in day; invalid 400
Release only, cut-off only, both, neither	Accepted; no relative order constraint between rules
Inactive supplier/user or reassigned user on write	401; preserve form and restore session
Inactive/missing hotel or unapproved/suspended link	404 for writes/room picker; owned contract reads remain available
Cross-supplier contract ID	404; no disclosure
Admin without exact route permission	403; super admin bypass still respects ownership/eligibility
Form/text Content-Type on write	415; use application/json
metadata?x=1, detail?page=1, list?sortOrder=asc	400 unsupported query
custom missing date; dates without custom	400, including everything+custom with missing dates
Repeated periods vs comma-joined/unknown values	Repeated accepted; invalid values400
minCapacity>maxCapacity or0/100	400; valid bounds1..99 inclusively compare maxOccupancy
Empty trimmed search/city/hotelName	Accepted, no effective text filter

Area / input	Expected outcome
starRating 1..5, validISOalpha2country, exact view	Valid; invalid values400; no-match valid filter gives empty results
page 0/limit 101/repeated scalar query	400; beyond-final positive page gives[]with total retained
Historical unknown/duplicate meals or mixed rate layouts	Historical validator rejects; current pricing route itself404
fixedPricing or top-level room/meal fields	400 unknown fields; submit the canonical pricing object on POST/PATCH
pricing null, duplicate pairs/meals, missing costs/bases, invalid scope/layout	400; new saves must be complete
Replacement invalidates a saved seasonal override	400; correct or remove the override in the same PATCH

Error envelope examples

Validation errors expressed as arrays produce the first string as message and all strings in data.errors. String-based conflicts and many not-found errors produce data:null. Representative full responses:

```
{
  "status": "error",
  "message": "onRequestSettings requires an On Request booking, release or sold-out policy",
  "data": {
    "errors": ["onRequestSettings requires an On Request booking, release or sold-out policy"]
  }
}

{
  "status": "error",
  "message": "Contract was updated; reload it and retry with the current version",
  "data": null
}
```

Unexpected database errors are not guaranteed400: the global filter returns status:error, message:Internal server error and data:null for internal failures, normally HTTP 500. Do not interpret every constraint exception as a frontend validation code. Treat offline/timeouts and non-JSON proxy responses separately.

16. Database invariants and frontend recovery

Retained storage is not an extra HTTP interface

Storage area	Invariants and integration relevance
supplier_hotel_contracts	Trimmed nonblank name, date order, uppercase3letter currency format, known status/model/booking type, version>=1, supplier/hotel/audit FKs. HTTP additionally validates real dates and ISO currency codes
Weekly parent columns	Null/same_all_week need empty weekend_days; split needs1..6unique known non-null names; one-dimensional array
supplier_hotel_contract_variations	Unique contract+room+view; matching hotel/room FKs, base/supplement/fixed kind, full-rate layout and amount bounds
Base supplements	One scalar amount for new writes; unequal migrated split values are preserved and require review
supplier_hotel_contract_meal_prices	Unique meal type within shared or variation scope; four upgrade codes; price and basis may be null only for imported incomplete data
Pricing completeness triggers	Enforce either model, rate layout, meal scope, imported-only incompleteness and room-only price bounds under the parent lock; preserve currency locking
Child-rule rows	Unique category/contract, ages0..17, from<=to, nonnegative numeric price; HTTP additionally caps price at10000000 and checks hotel child ceiling
Restriction rows	Ordered dates, minimum nights, weekday arrays, room/audit FKs; same-hotel membership is a service check. Term containment and active-room check are service rules
Season rows	Ordered dates and name/color constraints. Non-overlap/term containment are service checks under the parent lock; there is no database exclusion constraint
Seasonal price columns/rows	Optional base/child amounts, normalized variation/meal overrides with same-contract FKs and numeric bounds. Service rejects incompatible retained overrides atomically
Cancellation rows	Unique threshold/no-show shape, charge constraints, contract/season relations. Service validates full policy/count/value combinations
Release/cut-off columns	All null=absent; otherwise complete variant/time/action or cut-off pair. Postgres time serialized as HH:MM
SLA columns	Both null or complete10..120minute/action pair; eligibility includes booking type, release switch or sold-out-to-request, including explicit SQL null handling
Inventory rules	Parent mode/pool/policy plus normalized same-contract variation rows. Layout, bounds, full per-type coverage and non-draft completeness are constrained; finalization generates immutable nightly stock separately

Postgres numeric(12,2) can round direct SQL inputs; this is not equivalent to strict HTTP precision validation. Do not bypass APIs to initialize missing features. Migration-installed pricing triggers/functions are not guaranteed by TypeORM synchronize. Season overlap is checked by the service, not a database exclusion constraint. Use the existing migration deployment process. Run the new migration with DB_SYNC=false before deploying the API; implementation verification uses isolated test databases.

Access, locking and consistency

Create/update recheck active supplier, active user assigned to supplier, active hotel and approved link in one transaction. Supplier/hotel/link checks use read locks; PATCH locks the parent for writing and checks expectedVersion under the lock. Concurrent same-version writers serialize: one succeeds, the other receives409. A failed section rolls back earlier writes. An accepted no-op replacement still advances the shared version once; maximum version fails 409. Audit IDs are server assigned. Replacing restriction/season arrays recreates rows/audits, not individual field patches.

Detail uses repeatable-read transaction isolation so all returned sections share a snapshot. List/detail remain readable for an owned contract after hotel/link unavailability. Write/room-picker eligibility differs from read visibility. Lists omit detailed pricing/child/restriction/season/cancellation sections; retain basic settings, release/cut-off/SLA and hasPricing. hasPricing is based on stored weekPricingMode, not an on-demand integrity check of every row.

Frontend state and recovery procedure

1. Load metadata and eligible hotels. On hotel change discard incompatible rooms/reset picker pagination. Keep caches supplier-scoped and clear on account switch.

2. Keep saved detail/version separate from editable fields; track dirty sections to preserve omission semantics. Never spread responses into writes.

3. Validate serialized JSON numbers, complete dependent objects and enum codes. Localized labels are display-only.

4. Serialize form saves, disable duplicate submit, cancel stale searches, and replace saved version after every success. Preserve editable values after errors.

5. On409 reload detail/show differences. Do not automatically replay a stale full replacement over another editor's work.

6. On timeout reconcile GET detail before retrying PATCH. Create has no idempotency key; a blind retry can duplicate a draft. Inspect the list and explain uncertain outcome.

7. Display seasonal issues and effective prices/policy alongside stored overrides. Incomplete drafts do not imply activation/booking readiness.

8. Show incomplete migrated meal costs/bases and legacy supplements for review. Submit complete canonical pricing to resolve them; do not assume missing meal costs are zero.

Verified validation limitations

The installed class-validator IsNumber({maxDecimalPlaces:2}) implementation throws a TypeError for very small exponential values such as `1e-7`. This was reproduced with ChildPricingCategoryDto.price and SeasonChildPricesDto.extra_bed. Such requests can reach the global HTTP 500 handler instead of a clean 400; reject invalid precision in the frontend and preserve form input on internal errors. The separate room-price and cancellation validators do not use that decimal-count implementation. This audit records the limitation; no validator behavior was changed.

The parent release CHECK intends an all-null or complete policy, but uses nullable SQL IN expressions without an explicit non-null action check. For direct SQL, a missing release_after_action can make the CHECK result UNKNOWN, which PostgreSQL accepts. HTTP DTO validation still requires a valid action; do not treat database checks as a complete substitute for the API. The extended SLA check explicitly uses IS NOT NULL and IS TRUE, so its configured pair does not have that null loophole.

17. Complete field dictionary and schema coverage

Generated from the current OpenAPI schemas. Model-specific requirements are also enforced by the service; section 20 describes the pricing rules. Historical DTOs remain reference only.

ContractPublicationDto

Field	Type	Presence	Rules
`id`	integer	Required	
`timezone`	string	Required	
`finalizedAt`	string	Required	

InventoryCapDetailDto

Field	Type	Presence	Rules
`roomId`	integer	Required	minimum=1; maximum=2147483647
`view`	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
`variationId`	integer	Required	
`maxRoomsPerNight`	integer	Required	minimum=0; maximum=2147483647

InventoryRoomDetailDto

Field	Type	Presence	Rules
`roomId`	integer	Required	minimum=1; maximum=2147483647
`view`	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
`variationId`	integer	Required	
`roomsPerNight`	integer	Required	minimum=0; maximum=2147483647
`overbookingLimitPerNight`	number	Optional	minimum=1; maximum=2147483647; nullable

InventorySoldOutDetailDto

Field	Type	Presence	Rules
`action`	string	Required	values: stop_sale, on_request, controlled_overbooking
`overbookingLimitPerNight`	number	Optional	minimum=1; maximum=2147483647; nullable

ContractInventoryDetailDto

Field	Type	Presence	Rules
`mode`	string	Required	values: shared_pool, per_room_type, free_sale
`roomsPerNight`	integer	Optional	minimum=0; maximum=2147483647
`roomCaps`	array of InventoryCapDetailDto	Optional	
`rooms`	array of InventoryRoomDetailDto	Optional	
`whenSoldOut`	InventorySoldOutDetailDto	Optional	

InventoryIssueDto

Field	Type	Presence	Rules
`path`	string	Required	
`code`	string	Required	values: overbooking_limit_required
`message`	string	Required	

OnRequestSettingsDto

Field	Type	Presence	Rules
`slaMinutes`	integer	Required	minimum=10; maximum=120
`expiryAction`	string	Required	Auto-reject or Escalate to Hoteliana. Stored configuration only; expiry execution is not implemented.; values: auto_reject, escalate_to_hoteliana

ContractWeekPricingDto

Field	Type	Presence	Rules
`mode`	string	Required	values: same_all_week, weekday_weekend
`weekendDays`	array of string	Optional	Required for weekday_weekend: 1..6 unique lowercase weekday names. Forbidden for same_all_week. Returned in Monday-to-Sunday order.; minItems=1; maxItems=6; uniqueItems=true; values: monday, tuesday, wednesday, thursday, friday, saturday, sunday

ReleasePolicyDetailDto

Field	Type	Presence	Rules
`type`	string	Required	values: same_day, number_of_days
`daysBeforeCheckIn`	integer	Required	Set only for number_of_days; null for same_day.; nullable
`releaseTime`	string	Required	Local time-of-day, HH:MM.
`afterReleaseAction`	string	Required	values: stop_sale, on_request

BookingCutoffDetailDto

Field	Type	Presence	Rules
`daysBeforeCheckIn`	integer	Required	
`cutoffTime`	string	Required	Local time-of-day, HH:MM.

ContractRateDto

Field	Type	Presence	Rules
`amount`	number	Optional	minimum=0; maximum=10000000; multipleOf=0.01
`weekday`	number	Optional	minimum=0; maximum=10000000; multipleOf=0.01
`weekend`	number	Optional	minimum=0; maximum=10000000; multipleOf=0.01

MealPriceDetailDto

Field	Type	Presence	Rules
`type`	string	Required	values: bed_and_breakfast, half_board, full_board, iftar_and_suhoor
`pricingBasis`	string	Required	Null only for migrated incomplete meals.; values: per_person, per_room; nullable
`price`	number	Required	Null is unknown, never free.; minimum=0; maximum=10000000; nullable

RoomVariationDetailDto

Field	Type	Presence	Rules
`roomId`	integer	Required	minimum=1; maximum=2147483647
`view`	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
`variationId`	integer	Required	
`rate`	ContractRateDto	Optional	
`supplement`	number	Optional	
`legacySupplement`	ContractRateDto	Optional	Migrated unequal supplements, read only. Replace with a scalar supplement on save.
`mealPlans`	array of MealPriceDetailDto	Optional	

PricingDetailDto

Field	Type	Presence	Rules
`weekPricing`	ContractWeekPricingDto	Required	
`baseRoom`	RoomVariationDetailDto	Optional	
`rooms`	array of RoomVariationDetailDto	Required	
`mealPlans`	array of MealPriceDetailDto	Optional	

PricingIssueDto

Field	Type	Presence	Rules
`path`	string	Required	
`code`	string	Required	values: meal_price_missing, meal_basis_missing, room_supplement_review_required, season_supplement_review_required, season_pricing_layout_mismatch
`message`	string	Required	

ResolvedRoomPriceDto

Field	Type	Presence	Rules
`variationId`	integer	Required	
`view`	string	Required	
`mealPlans`	array of MealPriceDetailDto	Required	
`roomId`	integer	Required	
`price`	ContractRateDto	Required	

ContractChildCategoryDto

Field	Type	Presence	Rules
`from`	integer	Required	
`to`	integer	Required	
`price`	number	Required	

ContractChildPricingDto

Field	Type	Presence	Rules
`sharing_parents`	ContractChildCategoryDto	Required	
`shared_bed`	ContractChildCategoryDto	Required	
`extra_bed`	ContractChildCategoryDto	Required	

ContractRestrictionDetailDto

Field	Type	Presence	Rules
id	integer	Required	
contractId	integer	Required	
roomId	integer	Required	Null means the restriction applies to all rooms.; nullable
appliesFrom	string	Required	
appliesTo	string	Required	
minNights	integer	Required	
appliesOn	string	Required	values: every_day, weekend_days
restrictedCheckinDays	array of number	Required	Weekday numbers (0=Sunday .. 6=Saturday) closed for check-in.
restrictedCheckoutDays	array of number	Required	Weekday numbers (0=Sunday .. 6=Saturday) closed for check-out.
isActive	boolean	Required	
createdBySupplierUserId	integer	Required	
updatedBySupplierUserId	integer	Required	
createdAt	string	Required	
updatedAt	string	Required	

ContractSeasonIssueDto

Field	Type	Presence	Rules
code	string	Required	values: outside_contract_period, season_pricing_layout_mismatch, season_pricing_room_not_in_contract
message	string	Required	

MealPriceOverrideDto

Field	Type	Presence	Rules
type	string	Required	values: bed_and_breakfast, half_board, full_board, iftar_and_suhoor
price	number	Required	minimum=0; maximum=10000000; multipleOf=0.01

SeasonVariationDetailDto

Field	Type	Presence	Rules
`roomId`	integer	Required	minimum=1; maximum=2147483647
`view`	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
`rate`	ContractRateDto	Optional	
`supplement`	number	Optional	
`legacySupplement`	ContractRateDto	Optional	
`mealPlans`	array of MealPriceOverrideDto	Optional	

SeasonChildPricesDetailDto

Field	Type	Presence	Rules
`sharing_parents`	number	Optional	
`shared_bed`	number	Optional	
`extra_bed`	number	Optional	

SeasonPricingDetailDto

Field	Type	Presence	Rules
`baseRate`	ContractRateDto	Optional	Omit to inherit the contract base rate.
`rooms`	array of SeasonVariationDetailDto	Optional	
`mealPlans`	array of MealPriceOverrideDto	Optional	
`childPrices`	SeasonChildPricesDetailDto	Required	Only the categories this season re-prices.

CancellationChargeDetailDto

Field	Type	Presence	Rules
`kind`	string	Required	values: free, nights, percent, fixed
`value`	number	Optional	Absent for free cancellation.

CancellationTierDetailDto

Field	Type	Presence	Rules
`daysBefore`	integer	Required	
`charge`	CancellationChargeDetailDto	Required	

CancellationPolicyDetailDto

Field	Type	Presence	Rules
`tiers`	array of CancellationTierDetailDto	Required	Sorted from the largest daysBefore.
`noShow`	CancellationChargeDetailDto	Required	

ContractSeasonDetailDto

Field	Type	Presence	Rules
id	integer	Required	
contractId	integer	Required	
name	string	Required	
startDate	string	Required	
endDate	string	Required	
nights	integer	Required	Inclusive night count.
color	string	Required	
issues	array of ContractSeasonIssueDto	Required	Empty when valid. outside_contract_period appears when the contract term was later changed without updating seasons; incompatible new room/meal overrides are rejected atomically. Migrated pricing issues are reported at contract level.
pricing	SeasonPricingDetailDto	Required	The overrides this season stores. Empty means base pricing.
resolvedRoomPrices	array of ResolvedRoomPriceDto	Required	Per-night room-only prices and meal components: seasonal override, else contract. Base supplements are added once; fixed rates are full prices. Null until pricing is configured.; nullable
resolvedChildPricing	object	Required	Extra-child pricing during the season: contract age bands with season prices where overridden. Null when the contract has no child pricing.; nullable
cancellationPolicy	object	Required	This season’s own cancellation policy; null when it has none.; nullable
effectiveCancellationPolicy	object	Required	Policy that applies on the season’s nights: the season policy, else the contract policy, else null.; nullable
cancellationPolicySource	string	Required	values: season, contract; nullable
createdBySupplierUserId	integer	Required	
updatedBySupplierUserId	integer	Required	
createdAt	string	Required	
updatedAt	string	Required	

HotelContractDetailDto

Field	Type	Presence	Rules
publicationStatus	string	Required	values: not_finalized, ready
publication	object	Required	Publication id, captured hotel timezone and finalizedAt; null before finalization. Booking APIs are not part of this release.; nullable
inventoryRules	object	Required	nullable
inventoryStatus	string	Required	values: not_configured, incomplete, complete
inventoryIssues	array of InventoryIssueDto	Required	
id	integer	Required	
supplierId	integer	Required	
hotelId	integer	Required	
name	string	Required	
startDate	string	Required	
endDate	string	Required	
currency	string	Required	
bookingType	string	Required	allotment_block = Allotment-block (Instant); on_request = On Request.; values: allotment_block, on_request
onRequestSettings	object	Required	SLA configuration or null when unset; optional in drafts. Expiry actions are not executed by this module.; nullable
pricingModel	string	Required	values: base_plus_supplements, fixed_per_room
status	string	Required	values: draft, active, paused, terminated, scheduled, expired
weekPricing	object	Required	Null until pricing is configured. Read-only projection of pricing.weekPricing, shared by all variations.; nullable
releasePolicy	object	Required	Release period; null when none is configured.; nullable
bookingCutoff	object	Required	Booking cut-off; null when none is configured.; nullable
pricing	object	Required	Canonical pricing; replaces historical basePricing. Null for an unpriced draft.; nullable
pricingStatus	string	Required	values: not_configured, incomplete, complete
pricingIssues	array of PricingIssueDto	Required	
resolvedRoomPrices	array of ResolvedRoomPriceDto	Required	nullable
childPricing	object	Required	Extra-child pricing (three categories). Null when none is configured.; nullable
restrictions	array of ContractRestrictionDetailDto	Required	Booking restrictions on this contract (detail reads only; empty when none).

Field	Type	Presence	Rules
seasons	array of ContractSeasonDetailDto	Required	Rate seasons in date order (detail reads only; empty when none).
cancellationPolicy	object	Required	Contract cancellation policy; null when absent. Finalization requires effective cancellation/no-show coverage for every night.; nullable
createdBySupplierUserId	integer	Required	
updatedBySupplierUserId	integer	Required	
createdAt	string	Required	
updatedAt	string	Required	
version	integer	Required	minimum=1

FinalizationIssueDto

Field	Type	Presence	Rules
code	string	Required	
severity	string	Required	values: error, warning
section	string	Required	
path	string	Required	
message	string	Required	
entityId	integer	Optional	

InventoryPreviewDto

Field	Type	Presence	Rules
startDate	string	Required	
endDate	string	Required	
nights	integer	Required	minimum=0
mode	string	Required	nullable
poolCount	integer	Required	minimum=0
poolNights	integer	Required	minimum=0
capNights	integer	Required	minimum=0
estimatedRecords	integer	Required	minimum=0
rules	object	Required	Agreed nightly capacities, caps and overbooking allowances; null until configured.; nullable

ContractConfirmationDto

Field	Type	Presence	Rules
`version`	string	Required	values: contract-finalization-v1
`text`	string	Required	

FinalizationReviewDto

Field	Type	Presence	Rules
`contract`	HotelContractDetailDto	Required	
`contractId`	integer	Required	
`version`	integer	Required	
`evaluatedAt`	string	Required	
`businessDate`	string	Required	nullable
`timezone`	string	Required	nullable
`targetStatus`	string	Required	values: active, scheduled; nullable
`canFinalize`	boolean	Required	
`issues`	array of FinalizationIssueDto	Required	
`inventoryPreview`	InventoryPreviewDto	Required	
`confirmation`	ContractConfirmationDto	Required	

FinalizationReviewResponseDto

Field	Type	Presence	Rules
`status`	string	Required	values: success
`message`	string	Required	
`data`	FinalizationReviewDto	Required	

FinalizeContractDto

Field	Type	Presence	Rules
`expectedVersion`	integer	Required	minimum=1; maximum=2147483646
`confirmationAccepted`	boolean	Required	values: true
`confirmationVersion`	string	Required	values: contract-finalization-v1

InventoryGenerationDto

Field	Type	Presence	Rules
`startDate`	string	Required	
`endDate`	string	Required	
`nights`	integer	Required	minimum=0
`mode`	string	Required	nullable
`poolCount`	integer	Required	minimum=0
`poolNights`	integer	Required	minimum=0
`capNights`	integer	Required	minimum=0
`estimatedRecords`	integer	Required	minimum=0

FinalizedContractDto

Field	Type	Presence	Rules
`contract`	HotelContractDetailDto	Required	
`generation`	InventoryGenerationDto	Required	

FinalizedContractResponseDto

Field	Type	Presence	Rules
`status`	string	Required	values: success
`message`	string	Required	
`data`	FinalizedContractDto	Required	

ContractMetadataOptionDto

Field	Type	Presence	Rules
`value`	string	Required	
`labelEn`	string	Required	
`labelAr`	string	Required	

ContractCatalogOptionDto

Field	Type	Presence	Rules
`id`	integer	Required	
`code`	string	Required	
`nameEn`	string	Required	
`nameAr`	string	Required	

ContractMetadataDataDto

Field	Type	Presence	Rules
`inventoryModes`	array of ContractMetadataOptionDto	Required	
`soldOutActions`	array of ContractMetadataOptionDto	Required	
`mealPricingBases`	array of string	Required	values: per_person, per_room
`mealPlans`	array of ContractMetadataOptionDto	Required	
`roomViews`	array of ContractMetadataOptionDto	Required	
`roomSizes`	array of ContractCatalogOptionDto	Required	
`bedTypes`	array of ContractCatalogOptionDto	Required	

ContractMetadataResponseDto

Field	Type	Presence	Rules
`status`	string	Required	values: success
`message`	string	Required	
`data`	ContractMetadataDataDto	Required	

ContractHotelOptionDto

Field	Type	Presence	Rules
id	integer	Required	
nameEn	string	Required	nullable
nameAr	string	Required	nullable
countryCode	string	Required	nullable
city	string	Required	nullable
starRating	number	Required	nullable
propertyType	string	Required	nullable
status	string	Required	values: active

ContractPageMetaDto

Field	Type	Presence	Rules
total	integer	Required	
page	integer	Required	
limit	integer	Required	
totalPages	integer	Required	

ContractHotelOptionsDataDto

Field	Type	Presence	Rules
hotels	array of ContractHotelOptionDto	Required	
meta	ContractPageMetaDto	Required	
emptyState	string	Required	values: no_hotels, no_results, ; nullable

ContractHotelOptionsResponseDto

Field	Type	Presence	Rules
status	string	Required	values: success
message	string	Required	
data	ContractHotelOptionsDataDto	Required	

ContractRoomOptionDto

Field	Type	Presence	Rules
id	integer	Required	
hotelId	integer	Required	
nameEn	string	Required	nullable
nameAr	string	Required	nullable
view	string	Required	nullable
maxOccupancy	integer	Required	
adults	integer	Required	
bedTypeId	number	Required	nullable
bedTypeCode	string	Required	nullable
roomSizeId	number	Required	nullable
roomSizeCode	string	Required	nullable
areaSqm	number	Required	nullable
status	string	Required	values: active

ContractRoomOptionsDataDto

Field	Type	Presence	Rules
rooms	array of ContractRoomOptionDto	Required	
meta	ContractPageMetaDto	Required	
emptyState	string	Required	values: no_rooms, no_results, ; nullable

ContractRoomOptionsResponseDto

Field	Type	Presence	Rules
status	string	Required	values: success
message	string	Required	
data	ContractRoomOptionsDataDto	Required	

InventoryCapDto

Field	Type	Presence	Rules
roomId	integer	Required	minimum=1; maximum=2147483647
view	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
maxRoomsPerNight	integer	Required	Hard cap, never increased by pool overbooking.; minimum=0; maximum=2147483647

InventoryRoomDto

Field	Type	Presence	Rules
roomId	integer	Required	minimum=1; maximum=2147483647
view	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
roomsPerNight	integer	Required	minimum=0; maximum=2147483647
overbookingLimitPerNight	integer	Optional	Per-variation controlled-overbooking allowance. Omit on a draft to flag incomplete rules.; minimum=1; maximum=2147483647

InventorySoldOutDto

Field	Type	Presence	Rules
action	string	Required	values: stop_sale, on_request, controlled_overbooking
overbookingLimitPerNight	integer	Optional	Shared controlled-overbooking allowance only. Omit on a draft to flag incomplete rules.; minimum=1; maximum=2147483647

ContractInventoryDto

Field	Type	Presence	Rules
mode	string	Required	values: shared_pool, per_room_type, free_sale
roomsPerNight	integer	Optional	Required for shared_pool only.; minimum=0; maximum=2147483647
roomCaps	array of InventoryCapDto	Optional	Shared pool only. Omitted pairs have no additional cap; [] means no caps.; maxItems=1001
rooms	array of InventoryRoomDto	Optional	Per-room-type only: exactly one row per configured room/view, including the base variation.; maxItems=1001
whenSoldOut	InventorySoldOutDto	Optional	Required for quantified inventory; forbidden for free sale.

BaseVariationDto

Field	Type	Presence	Rules
`roomId`	integer	Required	minimum=1; maximum=2147483647
`view`	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
`rate`	ContractRateDto	Required	

MealPriceDto

Field	Type	Presence	Rules
`type`	string	Required	values: bed_and_breakfast, half_board, full_board, iftar_and_suhoor
`price`	number	Required	minimum=0; maximum=10000000; multipleOf=0.01
`pricingBasis`	string	Required	values: per_person, per_room

PricedVariationDto

Field	Type	Presence	Rules
`roomId`	integer	Required	minimum=1; maximum=2147483647
`view`	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
`rate`	ContractRateDto	Optional	Fixed model only: full room-only rate.
`supplement`	number	Optional	Base model only: one supplement all week.; minimum=0; maximum=10000000; multipleOf=0.01
`mealPlans`	array of MealPriceDto	Optional	Required for fixed variations, including [] for room only. Forbidden for base variations.; maxItems=4

VariationPricingDto

Field	Type	Presence	Rules
`weekPricing`	ContractWeekPricingDto	Required	
`baseRoom`	BaseVariationDto	Optional	Required for base pricing; forbidden for fixed pricing.
`rooms`	array of PricedVariationDto	Required	maxItems=1000
`mealPlans`	array of MealPriceDto	Optional	Required shared base-model upgrades ([] = room only); forbidden for fixed pricing.; maxItems=4

ChildPricingCategoryDto

Field	Type	Presence	Rules
`from`	integer	Required	minimum=0; maximum=17
`to`	integer	Required	minimum=0; maximum=17
`price`	number	Required	Per child per night. 0 means this category is free.; minimum=0; maximum=10000000

ChildPricingDto

Field	Type	Presence	Rules
`sharing_parents`	ChildPricingCategoryDto	Required	
`shared_bed`	ChildPricingCategoryDto	Required	
`extra_bed`	ChildPricingCategoryDto	Required	

CreateRestrictionDto

Field	Type	Presence	Rules
`roomId`	integer	Optional	Room type this restriction applies to. Omit for all rooms.; minimum=1; maximum=2147483647; nullable
`appliesFrom`	string	Required	
`appliesTo`	string	Required	
`minNights`	integer	Required	minimum=1; maximum=365
`appliesOn`	string	Required	values: every_day, weekend_days
`restrictedCheckinDays`	array of number	Optional	Weekday numbers (0=Sunday .. 6=Saturday) closed for this action, with no duplicates. Omit for no closed days.; minItems=0; maxItems=7; uniqueItems=true
`restrictedCheckoutDays`	array of number	Optional	Weekday numbers (0=Sunday .. 6=Saturday) closed for this action, with no duplicates. Omit for no closed days.; minItems=0; maxItems=7; uniqueItems=true

SeasonVariationDto

Field	Type	Presence	Rules
roomId	integer	Required	minimum=1; maximum=2147483647
view	string	Required	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
rate	ContractRateDto	Optional	
supplement	number	Optional	minimum=0; maximum=10000000; multipleOf=0.01
mealPlans	array of MealPriceOverrideDto	Optional	maxItems=4

SeasonChildPricesDto

Field	Type	Presence	Rules
sharing_parents	number	Optional	Per child per night during the season. Omit to keep the contract price; the age band always comes from the contract.; minimum=0; maximum=10000000
shared_bed	number	Optional	Per child per night during the season. Omit to keep the contract price; the age band always comes from the contract.; minimum=0; maximum=10000000
extra_bed	number	Optional	Per child per night during the season. Omit to keep the contract price; the age band always comes from the contract.; minimum=0; maximum=10000000

SeasonPricingDto

Field	Type	Presence	Rules
mealPlans	array of MealPriceOverrideDto	Optional	Base-model shared meal price overrides; selection and basis inherit.; maxItems=4
baseRate	ContractRateDto	Optional	Season base rate in the contract weekly layout; requires pricing supplied in the same request or already saved.
rooms	array of SeasonVariationDto	Optional	Existing room/view overrides: scalar supplement for base pricing, full rate and/or meal prices for fixed pricing. Cannot add variations.; maxItems=1000
childPrices	SeasonChildPricesDto	Optional	

CancellationChargeDto

Field	Type	Presence	Rules
kind	string	Required	values: free, nights, percent, fixed
value	number	Optional	Omit for free. nights: whole 1..365; percent: >0..100 of the stay; fixed: >0..10000000 in the contract currency. Two decimals at most.

CancellationTierDto

Field	Type	Presence	Rules
daysBefore	integer	Required	Applies when cancelled at least this many days before check-in.; minimum=1; maximum=365
charge	CancellationChargeDto	Required	

CancellationPolicyDto

Field	Type	Presence	Rules
tiers	array of CancellationTierDto	Required	Unique daysBefore values; returned sorted from the largest.; minItems=1; maxItems=3
noShow	CancellationChargeDto	Required	Charge for a no-show and for cancelling inside the smallest tier.

ContractSeasonDto

Field	Type	Presence	Rules
name	string	Required	minLength=1; maxLength=100
startDate	string	Required	
endDate	string	Required	Last night of the season (inclusive).
color	string	Required	Calendar color #RRGGBB, normalized to uppercase.
pricing	SeasonPricingDto	Optional	Seasonal pricing overrides. Omit (or send {}) to price the season like the base contract.
cancellationPolicy	CancellationPolicyDto	Optional	Seasonal cancellation policy; wins on the season's nights. Omit to use the contract policy.

ReleasePolicyDto

Field	Type	Presence	Rules
type	string	Required	same_day releases on the check-in day; number_of_days releases daysBeforeCheckIn days ahead.; values: same_day, number_of_days
daysBeforeCheckIn	integer	Optional	Days before check-in the inventory is released. Required for number_of_days; omit for same_day.; minimum=1; maximum=365
releaseTime	string	Required	Local time-of-day the release happens, HH:MM (00:00..23:59).
afterReleaseAction	string	Required	What happens to inventory once released.; values: stop_sale, on_request

BookingCutoffDto

Field	Type	Presence	Rules
daysBeforeCheckIn	integer	Required	Days before check-in a booking may still be created (0 = up to the check-in day).; minimum=0; maximum=365
cutoffTime	string	Required	Latest local time-of-day on the cut-off day, HH:MM (00:00..23:59).

CreateHotelContractDto

Field	Type	Presence	Rules
inventoryRules	ContractInventoryDto	Optional	Inventory defaults; POST/PATCH does not generate stock. Finalization generates nightly stock atomically. Requires configured pricing variations.
pricing	VariationPricingDto	Optional	Complete model-specific room/view and meal pricing. Omit to retain on PATCH; null is rejected.
hotelId	integer	Required	minimum=1; maximum=2147483647
name	string	Required	maxLength=200; minLength=1
startDate	string	Required	
endDate	string	Required	
currency	string	Required	ISO 4217 code, trimmed and normalized to uppercase.
bookingType	string	Required	Booking Confirmation Type: allotment_block = Allotment-block (Instant); on_request = On Request.; values: allotment_block, on_request
onRequestSettings	OnRequestSettingsDto	Optional	Optional in drafts. Complete SLA settings when bookingType, releasePolicy.afterReleaseAction or inventoryRules.whenSoldOut.action is on_request. Omit for none; null is rejected on create.
pricingModel	string	Required	values: base_plus_supplements, fixed_per_room
childPricing	ChildPricingDto	Optional	Extra-child pricing. Optional; when present, all three categories (sharing_parents, shared_bed, extra_bed) are required.
restrictions	array of CreateRestrictionDto	Optional	Optional booking restrictions created with the contract. Each date range must fall within the contract term.; maxItems=100
seasons	array of ContractSeasonDto	Optional	Optional rate seasons created with the contract. Seasons must fall within the contract term and must not overlap each other (a night belongs to at most one season). Nights without a season use the base contract configuration.; maxItems=100
cancellationPolicy	CancellationPolicyDto	Optional	Optional contract cancellation policy (required before activation). Applies to every night without a seasonal policy.
releasePolicy	ReleasePolicyDto	Optional	Optional release period: when unused contracted inventory is released. Omit for none.
bookingCutoff	BookingCutoffDto	Optional	Optional booking cut-off: the latest a new booking may be created. Omit for none.

HotelContractResponseDto

Field	Type	Presence	Rules
`status`	string	Required	values: success
`message`	string	Required	
`data`	HotelContractDetailDto	Required	

HotelContractSummaryDto

Field	Type	Presence	Rules
publicationStatus	string	Required	values: not_finalized, ready
publication	object	Required	Publication id, captured hotel timezone and finalizedAt; null before finalization. Booking APIs are not part of this release.; nullable
inventoryStatus	string	Required	values: not_configured, incomplete, complete
id	integer	Required	
supplierId	integer	Required	
hotelId	integer	Required	
name	string	Required	
startDate	string	Required	
endDate	string	Required	
currency	string	Required	
bookingType	string	Required	allotment_block = Allotment-block (Instant); on_request = On Request.; values: allotment_block, on_request
onRequestSettings	object	Required	SLA configuration or null when unset; optional in drafts. Expiry actions are not executed by this module.; nullable
pricingModel	string	Required	values: base_plus_supplements, fixed_per_room
status	string	Required	values: draft, active, paused, terminated, scheduled, expired
weekPricing	object	Required	Null until pricing is configured. Read-only projection of pricing.weekPricing, shared by all variations.; nullable
releasePolicy	object	Required	Release period; null when none is configured.; nullable
bookingCutoff	object	Required	Booking cut-off; null when none is configured.; nullable
pricingStatus	string	Required	values: not_configured, incomplete, complete
createdBySupplierUserId	integer	Required	
updatedBySupplierUserId	integer	Required	
createdAt	string	Required	
updatedAt	string	Required	
version	integer	Required	minimum=1
hasPricing	boolean	Required	

ContractsListDataDto

Field	Type	Presence	Rules
`contracts`	array of HotelContractSummaryDto	Required	
`meta`	ContractPageMetaDto	Required	
`emptyState`	string	Required	values: no_contracts, no_results, ; nullable

HotelContractsListResponseDto

Field	Type	Presence	Rules
`status`	string	Required	values: success
`message`	string	Required	
`data`	ContractsListDataDto	Required	

UpdateRestrictionDto

Field	Type	Presence	Rules
`roomId`	integer	Optional	Room type this restriction applies to. Omit for all rooms.; minimum=1; maximum=2147483647; nullable
`appliesFrom`	string	Required	
`appliesTo`	string	Required	
`minNights`	integer	Required	minimum=1; maximum=365
`appliesOn`	string	Required	values: every_day, weekend_days
`restrictedCheckinDays`	array of number	Optional	Weekday numbers (0=Sunday .. 6=Saturday) closed for this action, with no duplicates. Omit for no closed days.; minItems=0; maxItems=7; uniqueItems=true
`restrictedCheckoutDays`	array of number	Optional	Weekday numbers (0=Sunday .. 6=Saturday) closed for this action, with no duplicates. Omit for no closed days.; minItems=0; maxItems=7; uniqueItems=true
`isActive`	boolean	Optional	Whether this restriction applies. Defaults to true.; default=true

UpdateHotelContractDto

Field	Type	Presence	Rules
name	string	Optional	maxLength=200; minLength=1
startDate	string	Optional	
endDate	string	Optional	
currency	string	Optional	ISO 4217 code, trimmed and normalized to uppercase.
bookingType	string	Optional	Booking Confirmation Type: allotment_block = Allotment-block (Instant); on_request = On Request.; values: allotment_block, on_request
inventoryRules	object	Optional	Complete replacement; omit to preserve, null clears only on drafts. Missing controlled-overbooking limits are permitted only on drafts and report incomplete status.; nullable
pricing	VariationPricingDto	Optional	Complete model-specific room/view and meal pricing. Omit to retain on PATCH; null is rejected.
expectedVersion	integer	Required	minimum=1; maximum=2147483647
onRequestSettings	object	Optional	Full replacement. Omit to preserve; null clears. Allowed for merged bookingType=on_request, releasePolicy.afterReleaseAction=on_request or inventoryRules.whenSoldOut.action=on_request. Saved settings are automatically cleared when no On Request path applies.; nullable
childPricing	ChildPricingDto	Optional	Replaces the contract extra-child pricing (all three categories required). Omit to leave it unchanged.
restrictions	array of UpdateRestrictionDto	Optional	Replaces all booking restrictions for this contract (activate/deactivate via each entry's isActive). Omit to leave them unchanged; [] clears them. Each date range must fall within the contract term.; maxItems=100
seasons	array of ContractSeasonDto	Optional	Replaces all rate seasons for this contract. Omit to leave them unchanged; [] clears them. Seasons must fall within the contract term and must not overlap each other (a night belongs to at most one season). Nights without a season use the base contract configuration.; maxItems=100
cancellationPolicy	object	Optional	Replaces the contract cancellation policy. Omit to keep it; null removes it.; nullable
releasePolicy	object	Optional	Replaces the release period. Omit to keep it; null removes it.; nullable
bookingCutoff	object	Optional	Replaces the booking cut-off. Omit to keep it; null removes it.; nullable

ContractPaginationDto

Field	Type	Presence	Rules
page	integer	Optional	minimum=1; maximum=2147483647; default=1
limit	integer	Optional	minimum=1; maximum=100; default=20

FilterHotelContractsDto

Field	Type	Presence	Rules
page	integer	Optional	minimum=1; maximum=2147483647; default=1
limit	integer	Optional	minimum=1; maximum=100; default=20
search	string	Optional	maxLength=200
hotelName	string	Optional	Literal substring of English or Arabic hotel name (case-insensitive).; maxLength=200
status	string	Optional	Stored status; independent of coverage dates.; values: draft, active, paused, terminated, scheduled, expired
periods	array of string	Optional	Repeat periods to match any selected period. Omitted or everything means all coverage dates. Today uses Asia/Riyadh; next 90 days includes today and day 90.; values: everything, covering_today, starting_next_90_days, ending_next_90_days, already_over, custom
dateFrom	string	Optional	Required with custom; inclusive overlap range start.
dateTo	string	Optional	Required with custom; inclusive overlap range end.
sortBy	string	Optional	default="newest_first"; values: newest_first, ending_soonest, hotel_name
hotelId	integer	Optional	minimum=1; maximum=2147483647
pricingModel	string	Optional	values: base_plus_supplements, fixed_per_room

FilterContractHotelsDto

Field	Type	Presence	Rules
page	integer	Optional	minimum=1; maximum=2147483647; default=1
limit	integer	Optional	minimum=1; maximum=100; default=20
search	string	Optional	maxLength=200
sortOrder	string	Optional	default="asc"; values: asc, desc
sortBy	string	Optional	default="nameEn"; values: id, nameEn, nameAr, countryCode, city, starRating, createdAt
countryCode	string	Optional	
city	string	Optional	maxLength=120
starRating	integer	Optional	minimum=1; maximum=5
propertyType	string	Optional	values: hotel, resort, aparthotel, serviced_apartment, hostel, guesthouse, motel, villa, other

FilterContractRoomsDto

Field	Type	Presence	Rules
page	integer	Optional	minimum=1; maximum=2147483647; default=1
limit	integer	Optional	minimum=1; maximum=100; default=20
search	string	Optional	maxLength=200
sortOrder	string	Optional	default="asc"; values: asc, desc
sortBy	string	Optional	default="nameEn"; values: id, nameEn, nameAr, maxOccupancy, adults, areaSqm, createdAt
view	string	Optional	values: sea, city, garden, pool, mountain, courtyard, street, landmark, partial_haram, haram, no_view
bedTypeId	integer	Optional	minimum=1; maximum=2147483647
roomSizeId	integer	Optional	minimum=1; maximum=2147483647
minCapacity	integer	Optional	minimum=1; maximum=99
maxCapacity	integer	Optional	minimum=1; maximum=99

EmptyContractQueryDto

Field	Type	Presence	Rules

ContractRestrictionResponseDto

Field	Type	Presence	Rules
status	string	Required	values: success
message	string	Required	
data	ContractRestrictionDetailDto	Required	

ContractRoomSupplementDto

Field	Type	Presence	Rules
roomId	integer	Required	minimum=1; maximum=2147483647
supplement	ContractRateDto	Required	

ContractBasePricingDto

Field	Type	Presence	Rules
`baseRoomId`	integer	Required	minimum=1; maximum=2147483647
`baseRate`	ContractRateDto	Required	
`mealPlans`	array of string	Required	minItems=1; maxItems=5; uniqueItems=true; values: room_only, bed_and_breakfast, half_board, full_board, iftar_and_suhoor
`rooms`	array of ContractRoomSupplementDto	Required	maxItems=1000

ReplaceContractPricingDto

Field	Type	Presence	Rules
`expectedVersion`	integer	Required	minimum=1; maximum=2147483647
`weekPricing`	ContractWeekPricingDto	Required	
`basePricing`	ContractBasePricingDto	Required	Base model only at this checkpoint. Fixed pricing is deferred.

18. Source and rule coverage matrix

Paths are repository-relative. Source symbols identify rule implementations. Section17 includes a source and property marker for every DTO field. Controller reachability takes precedence over the existence of an old DTO/helper.

Topic / field families	Authoritative source
Nine routes, strict inputs, permissions	src/suppliers/hotel-contracts/hotel-contracts.controller.ts: SupplierHotelContractsController and strict
All write/query and nested field rules	src/suppliers/hotel-contracts/dto/hotel-contracts-input.dto.ts; section 17 maps every property
All returned shapes/projections	dto/hotel-contracts-response.dto.ts plus service contractResponse/restrictionResponse/seasonResponse/withChildrenAndRestrictions
Merged terms, currency lock and version	hotel-contracts.service.ts: create/update/validateDates; hotel-contract.entity.ts
Enums, limits and meal labels	hotel-contracts.constants.ts; views/property types in src/hoteliana/hotels/hotel-library.validation.ts
Inventory rules and readiness	contract-inventory.ts; contract-inventory.schema.ts; inventory DTOs and entity; migration1793404800000
Meals and variation pricing	dto/variation-pricing.dto.ts; contract-variation-pricing.ts; contract-variation.entity.ts. Historical rules remain in contract-pricing.ts
Child ages/prices and hotel ceiling	service validateChildPricing/assertChildPricingWithinCeiling/insertChildRules; contract-child-rule.entity.ts
Restriction term, room, weekday and state rules	service validateRestrictionWindow/requireRestrictionRoom/insertRestrictions; contract-restriction.entity.ts and contract-restriction.schema.ts
Season order/overlap/warnings	service validateSeasons/seasonResponse; contract-season.entity.ts; migration1792886400000

Topic / field families	Authoritative source
Seasonal overrides/price calculations	contract-variation-pricing.ts validateVariationSeason/resolveVariationPrices; service insertSeasons; migration1793318400000
Cancellation values/sort/inheritance	contract-cancellation.ts; service validateCancellationPolicies/seasonResponse; contract-cancellation.entity.ts
Release/cut-off variants/clear/time mapping	contract-release-cutoff.ts; service validateReleasePolicy; parent entity/migration1793145600000
SLA eligibility/auto-clear	contract-on-request.ts; service create/update; entity/migrations1793232000000 and1793404800000
Coverage timezone and inclusive90days	contract-coverage-dates.ts; service list; filter DTO
Pickers/search/sort/pagination/empty state	service hotels/rooms/eligibleHotels/sort/pageData; response DTO projections
Roles/session/permissions	src/suppliers/guards/supplier-roles.guard.ts and supplier-permissions.guard.ts; auth/supplier-jwt.strategy.ts; service requireSession
IDs/media/error envelope	src/common/pipes/database-id.pipe.ts; interceptors/json-content-type.interceptor.ts; filters/http-exception.filter.ts
Persisted keys/checks/triggers	src/database/migrations/1792454400000 through1793404800000 and entity declarations; section 16 distinguishes HTTP/service/DB checks
Diagram and fixed room+meal+view	documents/Suppliers hotel contracts (No inventory) flow.drawio. Design reference, not executable authority
Executable docs and example consistency	hotel-contracts.documentation.spec.ts and hotel-contracts.reference-documentation.spec.ts; companion TypeScript files beside PDF builder
Earlier pricing implementation/reference	Git705c1a6 implementation, guide76e9e12, removal diffc8a3dab

For source rows without a full prefix, files are under src/suppliers/hotel-contracts. Restriction weekend scope uses the fixed design constant CONTRACT_WEEKEND_DAYS=[4,5] (Thursday/Friday); pricing.weekendDays uses named days independently. The API stores appliesOn but does not execute booking-time restriction evaluation. Do not infer that pricing weekend selection changes restriction weekends.

OpenAPI bounds do not cover all service rules: merged containment, pairwise overlap, saved-pricing prerequisites, duplicate room identities, calculated totals, per-kind charges, hotel child ceilings and merged SLA eligibility are documented separately. No schema row implies an extra endpoint.

19. Git history and content-loss audit

Verified provenance (Cairo local time)

Commit / recorded author	Cairo timestamp and change
705c1a6 - Ashraf Isack	22 October 2026, 12:27:27 +03:00. Introduced base room/meal storage, validation, route and examples
76e9e12 - Ashraf Isack	26 October 2026, 11:04:03 +03:00. Guide/PDF synced with seasonal prices/cancellation; retained pricing PUT, meals and both weekly layouts
c8a3dab - solimanElzwawy	26 October 2026, 15:27:36 +03:00. Title: remove put supplier and make child pricing optional. Removed pricing PUT/help, replacement service code and permission; made child pricing optional. Did not update full guide/PDF in that commit
f161d96 - Ashraf Isack	27 October 2026, 10:12:38 +03:00. SLA feature commit also changed guide/PDF: removed detailed pricing workflow/both layouts and shortened seasonal pricing; added SLA/release/cut-off/unpriced-draft documentation

Git records commit authorship/committer metadata. It does not prove which person or assistant edited each working-tree line. The PDF regenerated in the preceding SLA task included the reduced guide. Distinguish the earlier backend removal from the later documentation reduction: recovering reference detail does not restore backend functionality.

Content-loss evidence and recovery

Both the PDF at 76e9e12 and at f161d96 had 15 pages; page count did not detect the loss. Extracted term counts: mealPlans 6 -> 1, baseRoomId 4 -> 0, weekendDays 5 -> 1, same_all_week 3 -> 0, weekday_weekend 4 -> 0. This guide restores those topics with examples/calculations/validation/status labels. Some metadata meal text survived; the loss was primarily implementation workflow/reference content.

The base write API remains historical because c8a3dab removed runtime code. Fixed room pricing was already documented as unimplemented before both changes; these commits do not show removal of a previously working fixed API. The full current fixed draft and diagram variation concept are therefore documented separately.

Reproduce and prevent another silent reduction

Run git log --follow --date=iso-strict on the Markdown/PDF; inspect git show c8a3dab and git show f161d96; compare guide76e9e12. Preserve exact commit IDs/dates when attributing changes; do not infer editing identity from author names alone.

Future updates must check field coverage against live schemas, validate current requests through DTO/service rules, validate historical examples separately, verify response inheritance and arithmetic, then regenerate/render every PDF page. Check pricing/meal topics and navigable headings as well as layout. Current request and historical markers must remain distinct to prevent removed routes reappearing as executable Postman steps. The Markdown remains the canonical PDF source.

20. Current room/view and meal pricing

POST and PATCH accept optional `pricing`. Omission preserves pricing; a supplied object replaces the complete section; null is rejected. The immutable contract pricingModel selects the shape. PATCH uses expectedVersion and increments it once. The removed PUT remains unavailable.

Base pricing has one baseRoom and zero to 1000 additional room/view pairs in rooms. Additional rooms use one scalar supplement for every day. Fixed pricing has 1..1000 room/view pairs, each with a complete room-only rate and mealPlans array. Full rates follow weekPricing (amount, or weekday/weekend). Distinct views can reuse the same catalog roomId; duplicate pairs and repeating the base pair are rejected. Select views explicitly from metadata, including partial_haram. Room restrictions still apply to all views of their catalog roomId.

Base mealPlans are shared; fixed mealPlans belong to their variation. Required arrays may be empty, meaning room only. Room only is always available without a supplement and is never submitted as a priced upgrade. Meals use type, pricingBasis (per_room or per_person), and price. Select one meal package for a stay night; do not add all selected packages together. Meal prices are constant throughout the week. Per-person meals charge adults and children at the same rate, separately from child accommodation charges.

Nightly total = effective room-only price + meal price × (1 for per_room; adults + children for per_person). A 400/500 base plus 120 Haram supplement and 30 breakfast per person costs 580/680 for two guests. Fixed City 450 plus 35 breakfast per person costs 520 for two guests; fixed Haram 650 plus 120 half board per room costs 770. Arithmetic uses integer hundredths; zero is an explicit free upgrade. Currency is contract currency and locks after room pricing exists. No VAT calculation or booking quote endpoint is introduced.

Responses replace historical basePricing with canonical pricing, variationId, view, resolvedRoomPrices (room-only price plus effective meal components), pricingStatus, and pricingIssues. Never spread responses into requests: variationId, legacySupplement and all computed fields are read only. IDs are preserved for unchanged room/view pairs. Lists keep weekPricing, hasPricing and pricingStatus but omit detailed pricing and issues. Metadata also returns mealPricingBases.

Complete creation examples

Base all week - all meal types and both bases

```
{
  "hotelId": 10,
  "name": "Winter 2026 contract",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "base_plus_supplements",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
```

```
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {"mode": "same_all_week"},
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"amount": 400}
    },
    "rooms": [
      {"roomId": 101, "view": "haram", "supplement": 120}
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  }
}
```

Base split week - shared meals and seasonal overrides

```
{
  "hotelId": 10,
  "name": "Winter 2026 contract",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "base_plus_supplements",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [],
  "cancellationPolicy": {
    "tiers": [
```

```
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 400, "weekend": 500}
    },
    "rooms": [
      {"roomId": 101, "view": "haram", "supplement": 120}
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  }
}
```

Fixed all week - variation meals and free seasonal breakfast

```
{
  "hotelId": 10,
  "name": "Winter 2026 fixed draft",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "fixed_per_room",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
```

```
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {"mode": "same_all_week"},
    "rooms": [
      {
        "roomId": 101,
        "view": "city",
        "rate": {"amount": 450},
        "mealPlans": [
          {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 35}
        ]
      },
      {
        "roomId": 101,
        "view": "haram",
        "rate": {"amount": 650},
        "mealPlans": [
          {"type": "half_board", "pricingBasis": "per_room", "price": 120}
        ]
      }
    ]
  }
}
```

Fixed split week - independent variation prices

```
{
  "hotelId": 10,
  "name": "Winter 2026 fixed draft",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "fixed_per_room",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
```

```
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "rooms": [
      {
        "roomId": 101,
        "view": "city",
        "rate": {"weekday": 450, "weekend": 550},
        "mealPlans": [
          {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 35}
        ]
      },
      {
        "roomId": 101,
        "view": "haram",
        "rate": {"weekday": 650, "weekend": 750},
        "mealPlans": [
          {"type": "half_board", "pricingBasis": "per_room", "price": 120}
        ]
      }
    ]
  }
}
```

Base room only - no additional variations


```
{
  "hotelId": 10,
  "name": "Winter 2026 contract",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "base_plus_supplements",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {"mode": "same_all_week"},
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"amount": 400}
    },
    "rooms": [],
    "mealPlans": []
  }
}
```

Fixed room only - partial Haram variation

```
{
  "hotelId": 10,
  "name": "Winter 2026 fixed draft",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "on_request",
  "onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
  "pricingModel": "fixed_per_room",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {"mode": "same_all_week"},
    "rooms": [
      {
        "roomId": 101,
        "view": "partial_haram",
        "rate": {"amount": 500},
        "mealPlans": []
      }
    ]
  }
```

```
}
```

Atomic replacement and seasonal examples

Seasons inherit every omitted component. Base seasons may override baseRate, scalar room/view supplements and shared mealPlans prices. Fixed seasons may override a room/view rate, its meal prices, or both. Meal selections and pricingBasis always inherit from the contract; seasons cannot add rooms or meals. Child prices remain independent and inherit contract age bands. Seasonal prices replace the matching component; supplements are added once.

Base split week - shared meals and seasonal overrides

```
{
  "expectedVersion": 1,
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 400, "weekend": 500}
    },
    "rooms": [
      {"roomId": 101, "view": "haram", "supplement": 120}
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  },
  "seasons": [
    {
      "name": "Peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "baseRate": {"weekday": 600, "weekend": 700},
        "rooms": [
          {"roomId": 101, "view": "haram", "supplement": 150}
        ],
        "mealPlans": [
          {"type": "bed_and_breakfast", "price": 40}
        ],
        "childPrices": {"extra_bed": 120}
      }
    }
  ]
}

{
  "status": "success",
  "message": "Contract updated successfully",
  "data": {
    "hotelId": 10,
    "name": "Winter 2026 contract",
    "startDate": "2026-11-01",
    "endDate": "2027-03-31",
    "currency": "EGP",
    "bookingType": "on_request",
    "pricingModel": "base_plus_supplements",
    "id": 501,
    "supplierId": 7,
    "status": "draft",
```

```
"version": 2,
"weekPricing": {
  "mode": "weekday_weekend",
  "weekendDays": ["friday", "saturday"]
},
"createdBySupplierUserId": 22,
"updatedBySupplierUserId": 22,
"createdAt": "2026-10-27T09:00:00.000Z",
"updatedAt": "2026-10-27T09:00:00.000Z",
"onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
"childPricing": {
  "sharing_parents": {"from": 0, "to": 5, "price": 0},
  "shared_bed": {"from": 6, "to": 11, "price": 50},
  "extra_bed": {"from": 6, "to": 11, "price": 100}
},
"cancellationPolicy": {
  "tiers": [
    {
      "daysBefore": 7,
      "charge": {"kind": "free"}
    },
    {
      "daysBefore": 3,
      "charge": {"kind": "nights", "value": 1}
    }
  ],
  "noShow": {"kind": "percent", "value": 100}
},
"releasePolicy": {
  "type": "number_of_days",
  "daysBeforeCheckIn": 7,
  "releaseTime": "18:00",
  "afterReleaseAction": "stop_sale"
},
"bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
"restrictions": [
  {
    "appliesFrom": "2026-11-01",
    "appliesTo": "2027-03-31",
    "minNights": 2,
    "appliesOn": "weekend_days",
    "restrictedCheckinDays": [0, 1],
    "restrictedCheckoutDays": [6],
    "roomId": null,
    "isActive": true,
    "id": 601,
    "contractId": 501,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
  },
  {
    "roomId": 102,
    "appliesFrom": "2026-12-20",
    "appliesTo": "2027-01-05",
    "minNights": 2,
    "appliesOn": "every_day",
    "restrictedCheckinDays": [5],
    "restrictedCheckoutDays": [6],
    "isActive": true,
    "id": 600,
    "contractId": 501,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
```

```
    }
  ],
  "seasons": [
    {
      "name": "Peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "baseRate": {"weekday": 600, "weekend": 700},
        "rooms": [
          {"roomId": 101, "view": "haram", "supplement": 150}
        ],
        "mealPlans": [
          {"type": "bed_and_breakfast", "price": 40}
        ],
        "childPrices": {"extra_bed": 120}
      },
      "id": 800,
      "contractId": 501,
      "nights": 17,
      "issues": [],
      "resolvedRoomPrices": [
        {
          "variationId": 1,
          "roomId": 101,
          "view": "city",
          "price": {"weekday": 600, "weekend": 700},
          "mealPlans": [
            {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 40},
            {"type": "full_board", "pricingBasis": "per_person", "price": 80},
            {"type": "half_board", "pricingBasis": "per_room", "price": 100},
            {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
          ]
        },
        {
          "variationId": 2,
          "roomId": 101,
          "view": "haram",
          "price": {"weekday": 750, "weekend": 850},
          "mealPlans": [
            {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 40},
            {"type": "full_board", "pricingBasis": "per_person", "price": 80},
            {"type": "half_board", "pricingBasis": "per_room", "price": 100},
            {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
          ]
        }
      ],
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 120}
      },
      "cancellationPolicy": null,
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 7,
            "charge": {"kind": "free"}
          },
          {
            "daysBefore": 3,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
```

```
      "cancellationPolicySource": "contract",
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T09:00:00.000Z",
      "updatedAt": "2026-10-27T09:00:00.000Z"
    }
  ],
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 400, "weekend": 500},
      "variationId": 1
    },
    "rooms": [
      {
        "roomId": 101,
        "view": "haram",
        "supplement": 120,
        "variationId": 2
      }
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  },
  "pricingStatus": "complete",
  "pricingIssues": [],
  "resolvedRoomPrices": [
    {
      "variationId": 1,
      "roomId": 101,
      "view": "city",
      "price": {"weekday": 400, "weekend": 500},
      "mealPlans": [
        {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
        {"type": "full_board", "pricingBasis": "per_person", "price": 80},
        {"type": "half_board", "pricingBasis": "per_room", "price": 100},
        {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
      ]
    },
    {
      "variationId": 2,
      "roomId": 101,
      "view": "haram",
      "price": {"weekday": 520, "weekend": 620},
      "mealPlans": [
        {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
        {"type": "full_board", "pricingBasis": "per_person", "price": 80},
        {"type": "half_board", "pricingBasis": "per_room", "price": 100},
        {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
      ]
    }
  ],
  "inventoryStatus": "not_configured",
  "inventoryRules": null,
  "inventoryIssues": [],
  "publicationStatus": "not_finalized",
  "publication": null
}
```

```
}
```

Fixed all week - variation meals and free seasonal breakfast

```
{
  "expectedVersion": 1,
  "pricing": {
    "weekPricing": {"mode": "same_all_week"},
    "rooms": [
      {
        "roomId": 101,
        "view": "city",
        "rate": {"amount": 450},
        "mealPlans": [
          {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 35}
        ]
      },
      {
        "roomId": 101,
        "view": "haram",
        "rate": {"amount": 650},
        "mealPlans": [
          {"type": "half_board", "pricingBasis": "per_room", "price": 120}
        ]
      }
    ]
  },
  "seasons": [
    {
      "name": "Peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "rooms": [
          {
            "roomId": 101,
            "view": "city",
            "rate": {"amount": 550},
            "mealPlans": [
              {"type": "bed_and_breakfast", "price": 0}
            ]
          }
        ]
      }
    }
  ]
}

{
  "status": "success",
  "message": "Contract updated successfully",
  "data": {
    "hotelId": 10,
    "name": "Winter 2026 fixed draft",
    "startDate": "2026-11-01",
    "endDate": "2027-03-31",
    "currency": "EGP",
    "bookingType": "on_request",
    "pricingModel": "fixed_per_room",
    "id": 502,
    "supplierId": 7,
    "status": "draft",
    "version": 2,
    "weekPricing": {"mode": "same_all_week"},
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
```

```
"createdAt": "2026-10-27T09:00:00.000Z",
"updatedAt": "2026-10-27T09:00:00.000Z",
"onRequestSettings": {"slaMinutes": 30, "expiryAction": "auto_reject"},
"childPricing": {
  "sharing_parents": {"from": 0, "to": 5, "price": 0},
  "shared_bed": {"from": 6, "to": 11, "price": 50},
  "extra_bed": {"from": 6, "to": 11, "price": 100}
},
"cancellationPolicy": {
  "tiers": [
    {
      "daysBefore": 7,
      "charge": {"kind": "free"}
    },
    {
      "daysBefore": 3,
      "charge": {"kind": "nights", "value": 1}
    }
  ],
  "noShow": {"kind": "percent", "value": 100}
},
"releasePolicy": {
  "type": "number_of_days",
  "daysBeforeCheckIn": 7,
  "releaseTime": "18:00",
  "afterReleaseAction": "stop_sale"
},
"bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
"restrictions": [
  {
    "appliesFrom": "2026-11-01",
    "appliesTo": "2027-03-31",
    "minNights": 2,
    "appliesOn": "weekend_days",
    "restrictedCheckinDays": [0, 1],
    "restrictedCheckoutDays": [6],
    "roomId": null,
    "isActive": true,
    "id": 611,
    "contractId": 502,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
  },
  {
    "roomId": 102,
    "appliesFrom": "2026-12-20",
    "appliesTo": "2027-01-05",
    "minNights": 2,
    "appliesOn": "every_day",
    "restrictedCheckinDays": [5],
    "restrictedCheckoutDays": [6],
    "isActive": true,
    "id": 610,
    "contractId": 502,
    "createdBySupplierUserId": 22,
    "updatedBySupplierUserId": 22,
    "createdAt": "2026-10-27T09:00:00.000Z",
    "updatedAt": "2026-10-27T09:00:00.000Z"
  }
],
"seasons": [
  {
    "name": "Peak",
    "startDate": "2026-12-20",
    "endDate": "2027-01-05",
```

```
      "color": "#E4572E",
      "pricing": {
        "rooms": [
          {
            "roomId": 101,
            "view": "city",
            "rate": {"amount": 550},
            "mealPlans": [
              {"type": "bed_and_breakfast", "price": 0}
            ]
          }
        ],
        "childPrices": {}
      },
      "id": 800,
      "contractId": 502,
      "nights": 17,
      "issues": [],
      "resolvedRoomPrices": [
        {
          "variationId": 1,
          "roomId": 101,
          "view": "city",
          "price": {"amount": 550},
          "mealPlans": [
            {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 0}
          ]
        },
        {
          "variationId": 2,
          "roomId": 101,
          "view": "haram",
          "price": {"amount": 650},
          "mealPlans": [
            {"type": "half_board", "pricingBasis": "per_room", "price": 120}
          ]
        }
      ],
      "resolvedChildPricing": {
        "sharing_parents": {"from": 0, "to": 5, "price": 0},
        "shared_bed": {"from": 6, "to": 11, "price": 50},
        "extra_bed": {"from": 6, "to": 11, "price": 100}
      },
      "cancellationPolicy": null,
      "effectiveCancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 7,
            "charge": {"kind": "free"}
          },
          {
            "daysBefore": 3,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      },
      "cancellationPolicySource": "contract",
      "createdBySupplierUserId": 22,
      "updatedBySupplierUserId": 22,
      "createdAt": "2026-10-27T09:00:00.000Z",
      "updatedAt": "2026-10-27T09:00:00.000Z"
    }
  ],
  "pricing": {
    "weekPricing": {"mode": "same_all_week"},
    "rooms": [
```

```
        {
          "roomId": 101,
          "view": "city",
          "rate": {"amount": 450},
          "mealPlans": [
            {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 35}
          ],
          "variationId": 1
        },
        {
          "roomId": 101,
          "view": "haram",
          "rate": {"amount": 650},
          "mealPlans": [
            {"type": "half_board", "pricingBasis": "per_room", "price": 120}
          ],
          "variationId": 2
        }
      ]
    },
    "pricingStatus": "complete",
    "pricingIssues": [],
    "resolvedRoomPrices": [
      {
        "variationId": 1,
        "roomId": 101,
        "view": "city",
        "price": {"amount": 450},
        "mealPlans": [
          {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 35}
        ]
      },
      {
        "variationId": 2,
        "roomId": 101,
        "view": "haram",
        "price": {"amount": 650},
        "mealPlans": [
          {"type": "half_board", "pricingBasis": "per_room", "price": 120}
        ]
      }
    ],
    "inventoryStatus": "not_configured",
    "inventoryRules": null,
    "inventoryIssues": [],
    "publicationStatus": "not_finalized",
    "publication": null
  }
}
```


Validation, conflicts and incomplete migration

Unknown fields, numeric strings, null prices, unsupported views/meals, duplicate room/view pairs or meal types, incorrect model fields, and mixed weekly layouts return 400. All amounts and calculated room-only rates are bounded by 10000000 with two decimals. Rooms must be active in the contract hotel under an eligible supplier link. A stale expectedVersion returns 409; reload and deliberately reapply edits. All failed writes roll back other supplied sections.

If replacement pricing removes a room/meal used by a saved season or changes an incompatible full-rate layout, return a field error and retain the prior version. Correct or remove the seasonal override in the same PATCH (seasons is a full array replacement; [] clears it). Omitting pricing retains it. Changed base selection is allowed only with a complete pricing replacement.

Migration converts old room/view rates and seasons, retaining missing meal prices/bases as null and marking pricingStatus=incomplete. Old unequal supplements appear in legacySupplement; they are never averaged or silently made equal. They still resolve to their exact old room-only prices, but their new save must use one scalar supplement. Missing meal

values never produce a meal-inclusive total. Fill every missing meal value and correct legacy seasonal supplements in a complete pricing/season save. Ordinary contract edits remain available while incomplete. pricingStatus=not_configured means no pricing; complete means all configuration values are known.

Deploy with DB_SYNC=false and run the appended migration before starting the new API. Superseded pricing tables are retained as *_legacy rollback archives, never read or written by the API. Migration aborts rather than dropping historical seasonal overrides whose rooms are no longer in the contract. Correct those stale references before retrying. Rollback permits unchanged migrated pricing and refuses changes requiring explicit conversion. Do not use synchronize to install completeness triggers.

Frontend acceptance and next checkpoint

Render the base room as locked in the room table, edit its full rate separately, and show shared meal prices once. For fixed rooms render view-specific meal selections and prices. Disable price-dependent totals when required migrated values are unknown, show pricingIssues next to the relevant field, and keep ordinary editing available. Preserve loaded pricing unless the user intentionally saves the complete section.

Review all scenarios after implementation verification. The proposed next pricing checkpoint is daily rate-calendar overrides; it requires a separate choice and implementation. Inventory defaults are described in section 21; finalization and nightly generation are implemented in section 22. Daily overrides and booking allocation remain separate work.

21. Contract inventory rules

InventoryRules stores contract defaults. Finalization generates nightly inventory atomically as described in section 22. Booking, inventory-calendar controls and consumption APIs are not included. Both pricing models support all three inventory modes independently.

Shared pool: one roomsPerNight quantity across all room/view variations. Optional roomCaps are hard limits and are never increased by overbooking. Pool 50 plus allowance 2 means maximum confirmed 52 across the pool, while a capped Deluxe variation remains at 12. Omitted caps mean no additional cap; caps need not sum to the pool quantity and may exceed it. All meal plans for the same variation share stock.

Per room type means per contract room/view pair, not catalog roomId alone. Every configured variation, including the base, appears exactly once with roomsPerNight. Stock and any controlled-overbooking allowance are independent per variation; a Deluxe booking cannot consume Standard stock. Zero quantities and zero shared caps are valid.

Free sale has no quantity, caps, sold-out point or overbooking setting. Submit only mode:free_sale. It supports either Instant or On Request bookingType. In the future calendar Open for sale allows sales and Stop sale blocks them; no unlimited numeric stock is created.

Write and response semantics

Optional inventoryRules on POST configures defaults against pricing in the same request. POST null is invalid. PATCH omit preserves; a supplied object fully replaces; null clears only on drafts. Use expectedVersion; combined pricing, inventory, seasons and other sections save atomically and increment the version once. Existing incomplete migrated pricing can still supply valid variation identities for inventory rules.

Quantified modes require whenSoldOut.action: stop_sale, on_request or controlled_overbooking. Shared controlled overbooking has one limit inside whenSoldOut; per-type controlled overbooking has one limit on each rooms entry. Limits are forbidden for other actions and at the wrong scope. Missing limits are allowed only on drafts, never mean unlimited, and must be completed before finalization. Supplied limits must be at least 1. Quantities/caps are integer 0..2147483647; limits are 1..2147483647 and quantity plus limit cannot exceed 2147483647. Up to 1001 unique same-contract room/view entries are allowed.

Detail returns inventoryRules:null and inventoryStatus:not_configured when absent, otherwise complete or incomplete plus inventoryIssues. Missing controlled limits appear as null in responses, with paths such as inventoryRules.whenSoldOut.overbookingLimitPerNight or inventoryRules.rooms[0].overbookingLimitPerNight. Null limit values and variationId are response-only: omit missing limits in draft writes rather than spreading the response. Lists include inventoryStatus but omit inventoryRules/inventoryIssues. Status measures inventory-rule completeness, not overall

finalization readiness. Metadata adds inventoryModes and soldOutActions with localized labels.

Sold-out-to-request reuses onRequestSettings even for Instant contracts. SLA eligibility uses the final merged bookingType, release policy and inventory policy. Omitted settings are retained while any request path remains, and automatically cleared when the last path disappears; explicitly incompatible settings return 400. Free sale alone does not create SLA eligibility. Settings remain optional on drafts; finalization requires them for request paths.

Complete contract creation examples

The following examples use the same configured roomId with separate city/haram variations. Replace hotel/room IDs with eligible values from the pickers. Postman provides all policies under both pricing models as saved original requests and responses.

base_plus_supplements: Shared pool: stop_sale

```
{
  "hotelId": 10,
  "name": "Inventory Shared pool: stop_sale",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "allotment_block",
  "pricingModel": "base_plus_supplements",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      }
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
```

```
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      }
    }
  ],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 400, "weekend": 500}
    },
    "rooms": [
      {"roomId": 101, "view": "haram", "supplement": 120}
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  },
  "inventoryRules": {
    "mode": "shared_pool",
    "roomsPerNight": 50,
    "roomCaps": [
      {"roomId": 101, "view": "haram", "maxRoomsPerNight": 12}
    ],
    "whenSoldOut": {"action": "stop_sale"}
  }
}
```

base_plus_supplements: Per room/view: controlled_overbooking

```
{
  "hotelId": 10,
  "name": "Inventory Per room/view: controlled_overbooking",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "allotment_block",
  "pricingModel": "base_plus_supplements",
```

```
"childPricing": {
  "sharing_parents": {"from": 0, "to": 5, "price": 0},
  "shared_bed": {"from": 6, "to": 11, "price": 50},
  "extra_bed": {"from": 6, "to": 11, "price": 100}
},
"restrictions": [
  {
    "roomId": 102,
    "appliesFrom": "2026-12-20",
    "appliesTo": "2027-01-05",
    "minNights": 2,
    "appliesOn": "every_day",
    "restrictedCheckinDays": [5],
    "restrictedCheckoutDays": [6]
  },
  {
    "appliesFrom": "2026-11-01",
    "appliesTo": "2027-03-31",
    "minNights": 2,
    "appliesOn": "weekend_days",
    "restrictedCheckinDays": [0, 1],
    "restrictedCheckoutDays": [6]
  }
],
"seasons": [
  {
    "name": "Christmas peak",
    "startDate": "2026-12-20",
    "endDate": "2027-01-05",
    "color": "#E4572E",
    "pricing": {
      "childPrices": {"sharing_parents": 0, "extra_bed": 120}
    }
  },
  {
    "name": "February winter season",
    "startDate": "2027-02-01",
    "endDate": "2027-02-28",
    "color": "#2E86AB",
    "cancellationPolicy": {
      "tiers": [
        {
          "daysBefore": 14,
          "charge": {"kind": "nights", "value": 1}
        }
      ],
      "noShow": {"kind": "percent", "value": 100}
    }
  }
],
"cancellationPolicy": {
  "tiers": [
    {
      "daysBefore": 7,
      "charge": {"kind": "free"}
    },
    {
      "daysBefore": 3,
      "charge": {"kind": "nights", "value": 1}
    }
  ],
  "noShow": {"kind": "percent", "value": 100}
},
"releasePolicy": {
  "type": "number_of_days",
  "daysBeforeCheckIn": 7,
  "releaseTime": "18:00",
```

```
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 400, "weekend": 500}
    },
    "rooms": [
      {"roomId": 101, "view": "haram", "supplement": 120}
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  },
  "inventoryRules": {
    "mode": "per_room_type",
    "rooms": [
      {
        "roomId": 101,
        "view": "city",
        "roomsPerNight": 20,
        "overbookingLimitPerNight": 2
      },
      {
        "roomId": 101,
        "view": "haram",
        "roomsPerNight": 12,
        "overbookingLimitPerNight": 1
      }
    ],
    "whenSoldOut": {"action": "controlled_overbooking"}
  }
}
```

base_plus_supplements: Free sale with Instant

```
{
  "hotelId": 10,
  "name": "Inventory Free sale with Instant",
  "startDate": "2026-11-01",
  "endDate": "2027-03-31",
  "currency": "EGP",
  "bookingType": "allotment_block",
  "pricingModel": "base_plus_supplements",
  "childPricing": {
    "sharing_parents": {"from": 0, "to": 5, "price": 0},
    "shared_bed": {"from": 6, "to": 11, "price": 50},
    "extra_bed": {"from": 6, "to": 11, "price": 100}
  },
  "restrictions": [
    {
      "roomId": 102,
      "appliesFrom": "2026-12-20",
      "appliesTo": "2027-01-05",
      "minNights": 2,
      "appliesOn": "every_day",
      "restrictedCheckinDays": [5],
      "restrictedCheckoutDays": [6]
```

```
    },
    {
      "appliesFrom": "2026-11-01",
      "appliesTo": "2027-03-31",
      "minNights": 2,
      "appliesOn": "weekend_days",
      "restrictedCheckinDays": [0, 1],
      "restrictedCheckoutDays": [6]
    }
  ],
  "seasons": [
    {
      "name": "Christmas peak",
      "startDate": "2026-12-20",
      "endDate": "2027-01-05",
      "color": "#E4572E",
      "pricing": {
        "childPrices": {"sharing_parents": 0, "extra_bed": 120}
      }
    },
    {
      "name": "February winter season",
      "startDate": "2027-02-01",
      "endDate": "2027-02-28",
      "color": "#2E86AB",
      "cancellationPolicy": {
        "tiers": [
          {
            "daysBefore": 14,
            "charge": {"kind": "nights", "value": 1}
          }
        ],
        "noShow": {"kind": "percent", "value": 100}
      }
    }
  ],
  "cancellationPolicy": {
    "tiers": [
      {
        "daysBefore": 7,
        "charge": {"kind": "free"}
      },
      {
        "daysBefore": 3,
        "charge": {"kind": "nights", "value": 1}
      }
    ],
    "noShow": {"kind": "percent", "value": 100}
  },
  "releasePolicy": {
    "type": "number_of_days",
    "daysBeforeCheckIn": 7,
    "releaseTime": "18:00",
    "afterReleaseAction": "stop_sale"
  },
  "bookingCutoff": {"daysBeforeCheckIn": 1, "cutoffTime": "14:00"},
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 400, "weekend": 500}
    },
    "rooms": [
```

```
      {"roomId": 101, "view": "haram", "supplement": 120}
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  },
  "inventoryRules": {"mode": "free_sale"}
}
```

Replacement and incomplete-draft examples

A replacement that removes a capped variation or changes per-type coverage must correct inventory in the same PATCH. Shared/free-sale rules cover newly added pricing variations automatically. Pricing variations and their stable IDs are not recreated for a rule-only change.

```
{
  "expectedVersion": 1,
  "pricing": {
    "weekPricing": {"mode": "same_all_week"},
    "rooms": [
      {
        "roomId": 101,
        "view": "city",
        "rate": {"amount": 450},
        "mealPlans": [
          {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 35}
        ]
      },
      {
        "roomId": 101,
        "view": "haram",
        "rate": {"amount": 650},
        "mealPlans": [
          {"type": "half_board", "pricingBasis": "per_room", "price": 120}
        ]
      }
    ]
  },
  "inventoryRules": {
    "mode": "per_room_type",
    "rooms": [
      {
        "roomId": 101,
        "view": "city",
        "roomsPerNight": 20,
        "overbookingLimitPerNight": 2
      },
      {
        "roomId": 101,
        "view": "haram",
        "roomsPerNight": 12,
        "overbookingLimitPerNight": 1
      }
    ],
    "whenSoldOut": {"action": "controlled_overbooking"}
  },
  "bookingType": "allotment_block",
  "onRequestSettings": null
}
```

Clear rules on a draft (or omit inventoryRules to retain them):

```
{"expectedVersion": 2, "inventoryRules": null}
```

Select controlled overbooking before entering its limit, on a draft only:

```
{
  "expectedVersion": 1,
  "pricing": {
    "weekPricing": {
      "mode": "weekday_weekend",
      "weekendDays": ["friday", "saturday"]
    },
    "baseRoom": {
      "roomId": 101,
      "view": "city",
      "rate": {"weekday": 400, "weekend": 500}
    },
    "rooms": [
      {"roomId": 101, "view": "haram", "supplement": 120}
    ],
    "mealPlans": [
      {"type": "bed_and_breakfast", "pricingBasis": "per_person", "price": 30},
      {"type": "half_board", "pricingBasis": "per_room", "price": 100},
      {"type": "full_board", "pricingBasis": "per_person", "price": 80},
      {"type": "iftar_and_suhoor", "pricingBasis": "per_room", "price": 0}
    ]
  },
  "inventoryRules": {
    "mode": "shared_pool",
    "roomsPerNight": 0,
    "whenSoldOut": {"action": "controlled_overbooking"}
  },
  "bookingType": "allotment_block",
  "onRequestSettings": null
}
```

Inventory fields from the resulting detail response (other contract fields omitted here):

```
{
  "inventoryRules": {
    "mode": "shared_pool",
    "roomsPerNight": 0,
    "whenSoldOut": {"action": "controlled_overbooking", "overbookingLimitPerNight": null},
    "roomCaps": []
  },
  "inventoryStatus": "incomplete",
  "inventoryIssues": [
    {
      "path": "inventoryRules.whenSoldOut.overbookingLimitPerNight",
      "code": "overbooking_limit_required",
      "message": "Enter an overbooking limit of at least 1 before finalization; missing never means unlimited"
    }
  ]
}
```

Error recovery and generation boundaries

Unknown fields, numeric strings, null or fractional quantities, duplicate room/view pairs, foreign variations, incomplete per-type coverage, invalid policy fields and overflow return 400. Stale versions return 409. Invalid combined edits roll back all sections. Inventory cannot be configured without pricing variations. No quantities are inferred from room occupancy or existing contracts during migration. New rules and SQL scope/layout constraints are installed by append-only migration 1793404800000 with DB_SYNC=false. Rollback refuses configured rules rather than silently dropping them.

Finalization generates dated pool rows and shared variation caps for inclusive stay nights (section 22). The immutable publication revision identifies source defaults; retries verify rather than reset stock. Meals share the variation stock. Free Sale has null capacity. Amendments, daily controls, timed holds and atomic booking allocation remain future boundaries; no booking guarantee or booking endpoint is added.

Review these scenarios together with section 22. The next inventory checkpoint is daily Open/Stop Sale controls and booking allocation.

22. Contract finalization

Updated: 1 November 2026. Finalization is enabled only after deployment checks. No Hoteliana approval step is introduced. Draft creation and editing remain available; published commercial configuration is immutable.

Review and correct

Call GET /suppliers/hotel-contracts/:id/finalization-review with the supplier bearer token and view permission. No query fields are accepted. It returns canonical contract detail, version, evaluation timestamp, hotel-local businessDate/timezone, targetStatus, canFinalize, issues, inventoryPreview and confirmation.

An issue has code, severity (error or warning), section, path, message and optional entityId. Show all independent errors grouped by section, with links to the named fields; never parse message text to identify a field. An empty issues array means the review found no issues. Every error blocks finalization. Review performs no writes.

Inventory preview contains startDate, endDate, nights, mode, poolCount, poolNights, capNights, estimatedRecords and rules. The rules contain nightly capacities, hard caps and overbooking allowances. A contract period is inclusive stay nights; a booking later consumes check-in through the night before checkout. The final contract night may therefore be followed by checkout on the next date.

The backend requires complete effective room/meal pricing, active eligible rooms/hotel/link/supplier, an explicit inventory mode, a valid hotel timezone, effective cancellation/no-show coverage on every contract night and SLA/expiry settings for every On Request path. Season gaps inherit base pricing. Child pricing, release, cutoff and restrictions remain optional; configured values must be valid. Zero inventory and zero caps are valid. Missing quantities never mean unlimited stock. Free Sale must be selected explicitly for unquantified supply, including unquantified On Request supply.

Finalization is bounded to 3,660 inclusive nights and 1,000,000 generated nightly records. Larger drafts remain editable and display blocking review issues.

Confirmation and submission

Display the exact server-owned confirmation text and retain its version. Require a deliberate checkbox acceptance. Use Schedule contract when targetStatus is scheduled, otherwise Activate contract when active. Do not claim that Scheduled means sales must wait for the stay period to begin: finalized future nights are eligible for advance booking integration.

POST /suppliers/hotel-contracts/:id/finalize requires suppliers.hotel_contracts.finalize, JSON content type and an Idempotency-Key UUID header. Existing administrator profiles are not automatically granted this permission; Super Admin keeps its normal access.

```
{
  "expectedVersion": 1,
  "confirmationAccepted": true,
  "confirmationVersion": "contract-finalization-v1"
}
```

Example header: Idempotency-Key: a2082097-e4cb-4747-a282-357ba2226683. Generate a new UUID once for each deliberate submission. On a network failure retain BOTH the key and exact body. A matching retry returns the original receipt, even if automatic lifecycle processing has since changed the current version. Refresh detail separately for the current state. Never increment expectedVersion while replaying an old request.

HTTP 200 data contains contract (canonical published detail) and generation (night/pool/cap record counts). contract.publicationStatus is ready, and publication contains id, the captured timezone and finalizedAt. Before finalization publicationStatus is not_finalized and publication is null. GET detail and list expose these fields as well.

HTTP 422 uses the existing error envelope with data.issues and data.version. For example:

```
{
  "status": "error",
  "message": "Resolve blocking issues before finalizing",
```

```
  "data": {
    "version": 1,
    "issues": [
      {
        "code": "inventory_required",
        "severity": "error",
        "section": "inventory",
        "path": "inventoryRules",
        "message": "Choose Shared Pool, Per Room Type or Free Sale."
      }
    ]
  }
}
```


HTTP 400 means malformed types/unknown fields/header; 409 means stale version, published state or incompatible key reuse; 503 means the feature is disabled. On 409 reload the review and ask for deliberate acceptance of the updated terms. Do not auto-finalize a newer revision.

Publication, stock and lifecycle

The transaction rechecks eligibility and completeness, captures commercial/room configuration and timezone, generates all nightly stock, records supplier confirmation, changes status and increments the version once. Failure rolls back everything. No contract becomes Active or Scheduled without complete stock.

Start Date in the future yields scheduled (Upcoming). A term covering the hotel-local business date yields active. Ended contracts cannot finalize. The worker checks at startup and each minute, advances eligible Scheduled contracts, expires Scheduled/Active contracts after their final night and catches up after downtime. Ineligible links/hotels/suppliers block activation with a logged reason. Paused/Terminated remain unchanged. Status changes increment version without rebuilding inventory.

The captured hotel timezone remains authoritative even if the library hotel timezone changes. Existing list coverage filters still use Asia/Riyadh. Release/cutoff times will use the captured hotel timezone when execution is implemented; this release stores those rules only.

Shared Pool uses one nightly quantity across variations. Pool 50 with allowance +2 permits a total 52, while a City View cap of 12 remains 12. Per Room Type gives each room/view its own stock. Free Sale has null capacity, not a numeric maximum. Meals never create additional pools. Every contract is a distinct supplier entitlement; hotel identity alone does not combine pools.

PATCH is rejected for non-Draft contracts. Database guards also prevent modifying published terms, defaults, stock or confirmation evidence. No amendment, daily override, regeneration, booking, timed hold or allocation endpoint is included. Publication readiness and stored status do not replace future booking-time eligibility/date/capacity/policy checks.

Legacy Active/Scheduled contracts without publication evidence return to Draft during migration with an audit record and version increment. Paused/Terminated/Expired history is preserved. Nothing is automatically confirmed or published.

Integration and deployment references

The checked-in contract-finalization-client.ts next to this guide provides strict review/finalize request helpers and structured issue handling. Retain the review and request key as submission state. A finalized contract is read-only in v1.

See contract-finalization-deployment-runbook.md for backup, read-only preflight, reviewed schema-sync reconciliation, compiled migration, feature flags, smoke tests and retained-image recovery. No application database is migrated during local acceptance.